Pilot-guided deep reinforcement learning for navigation of a jellyfish-like swimmer in flows with obstacles

Yihao Chen
State Key Laboratory for Turbulence and Complex Systems, School of Mechanics and Engineering Science, Peking University, Beijing 100871, China

Yue Yang*
State Key Laboratory for Turbulence and Complex Systems, School of Mechanics and Engineering Science, Peking University, Beijing 100871, China
and HEDPS-CAPT, Peking University, Beijing 100871, China

(Received 6 November 2025; accepted 15 September 2026; published 6 October 2026)

We develop a deep reinforcement learning framework for controlling a bio-inspired jellyfish swimmer to navigate complex fluid environments with obstacles. While existing deep reinforcement leaning frameworks for underwater navigation often use only kinematic and geometric state representations (e.g., position, orientation, velocity, and proximity distances), a key challenge remains in achieving efficient obstacle avoidance under strong fluid-structure interactions and near-wall effects. We augment the agent's state representation within a soft actor-critic algorithm to include the real-time forces and torque experienced by the swimmer, providing direct mechanical feedback from vortex-wall interactions. A path-planning module supplies a local pilot point along the computed shortest path to guide the swimmer, reducing the navigation task to reaching a nearby target while avoiding obstacles. This augmented state space enables the swimmer to perceive and interpret wall proximity and orientation through distinct hydrodynamic force signatures. We analyze how these force and torque patterns, generated by walls at different positions influence the swimmer's decision-making policy. Comparative experiments with a baseline model without force feedback demonstrate that the present one with force feedback achieves higher navigation efficiency in two-dimensional obstacle-avoidance tasks. The results show that explicit force feedback facilitates earlier, smoother maneuvers and enables the exploitation of wall effects for efficient turning behaviors. With an application to autonomous cave mapping, this work underscores the critical role of direct mechanical feedback in the reinforcement learning framework for improving navigation within complex fluid environments.

DOI: 10.1103/g3qy-d7q7

I. INTRODUCTION

Deep reinforcement learning (DRL) [1,2] has emerged as an important tool for tackling complex control challenges in fluid dynamics, particularly those involving intricate fluid-structure interactions (FSIs). Recent studies have demonstrated the effectiveness of DRL in controlling FSI systems, including transonic buffet suppression [3], vibration mitigation of flexible structures [4], and propulsion optimization of self-propelled swimmers [5]. DRL takes a different approach from traditional control strategies such as adjoint-based optimal control and linear feedback controllers,

*Contact author: yyg@pku.edu.cn

 ©2026 American Physical Society

which rely on accurate reduced-order models of the system dynamics. For many complex problems, including those with strong fluid-structure interaction, such models are difficult to construct. DRL learns control policies directly from data, bypassing the need for explicit dynamical models. It enables the autonomous learning of control policies that can adaptively stabilize, maneuver, or optimize fluid systems, demonstrating significant success in applications such as aerodynamic shape optimization [6–10], drag reduction [11–17], and active flow control [5,18–24].

In aerodynamic shape optimization, DRL has been employed to iteratively refine airfoil geometries for improved lift-to-drag ratios [6–8] and to explore nonintuitive design spaces beyond traditional parametrizations [9,10]. For drag reduction, studies have focused on bluff body wake control [11–13], turbulent boundary layer modulation [14,15], and feedback control of cylinder flows using sparse pressure measurements [17]. In active flow control, DRL agents have been trained to suppress separation [18–20], enhance mixing [21,22], and synchronize multiple swimmers for efficient propulsion [5,23,24]. Building on these foundations, recent work has extended DRL to point-to-point navigation in turbulent flows [25], turbulent drag reduction in channel flows [26], and autonomous underwater navigation where flow gradient sensing is critical [27]. These advances motivate the present study, which addresses the additional challenge of navigating complex, obstacle-laden environments under strong fluid-structure interactions.

A demanding frontier for DRL lies in the control of soft-bodied underwater robots navigating confined, obstacle-laden spaces, such as underwater caves or wreck interiors [28,29]. Soft robotic systems have operational safety advantage over traditional rigid robots in several aspects [30–32]. First, their compliant nature provides inherent collision resilience. The energy from unintended impacts with obstacles is absorbed and dissipated through the robot's body, drastically reducing the risk of mechanical damage and ensuring mission continuity. Furthermore, this compliance translates into superior maneuverability. Soft robots can safely navigate through complex and confined environments, such as coral reefs or underwater ruins, without the need for forceful interaction that could damage both the robot and the surroundings.

In these scenarios, agents face a dual challenge: the complex, often unintuitive dynamics of FSI inherent to flexible morphologies, and the significant hydrodynamic wall effects that dominate near boundaries [29,33–36]. These effects can drastically alter a swimmer's dynamics, making precise, robust control difficult. This challenge is compounded by the limitations of conventional sensing. Many current DRL strategies [37–41] for object motion control in fluid rely predominantly on kinematic or geometric state representations (e.g., position, velocity, proximity distances), often overlooking the direct physical force interactions between the compliant agent and its fluid environment as FSI does not have significant influence. For a soft robot, this limitation is critical, as its shape and the forces upon it are intrinsically linked, and traditional states may not be sufficient to capture the nuanced hydrodynamic cues essential for anticipating and mitigating collisions in tight spaces.

To bridge this gap, this work focuses on a fundamental aspect of physical interaction: the direct force and torque feedback experienced by a controlled soft body immersed in a fluid. We posit that for a compliant swimmer in a confined environment, incorporating these mechanical interaction signals as explicit state variables within the DRL framework provides essential, real-time information about boundary proximity and orientation, enabling more informed and anticipatory decision-making for navigation and control.

Specifically, we investigate this hypothesis using a bio-inspired, DRL-controlled 2D jellyfish swimmer [42] navigating obstacle-laden environments. Bio-inspired design shows advanced performance in various application [5,43]. Jellyfish exhibit high locomotion efficiency through body deformation and FSI [44,45]. Their swimming mechanics, characterized by energy-efficient propulsion [46–49] and morphological simplicity, enable effective maneuverability [42]. These traits make jellyfish-inspired robots promising for ocean exploration and ecosystem research [50–52].

Building upon prior work [53], our core contribution lies in augmenting the agent's state representation to include the forces and torque exerted by the surrounding fluid on the jellyfish body. While visual sensing of obstacle boundaries (e.g., via multidirectional distance measurements) has

proven effective in relatively simple environments [54], its generalization to cluttered, nonconvex underwater domains often demands high-dimensional state representations and complex network architectures, increasing training difficulty. In contrast, we argue that in strongly interactive fluid environments, the hydrodynamic forces and torque acting on a soft-bodied swimmer provide compact yet informative signals that encode wall proximity and orientation through vortex-wall interactions. These mechanical cues can be measured in practice using embedded pressure or force sensors, offering an experimentally feasible alternative to vision. Moreover, by combining force and torque feedback with a path-planning module that supplies a local pilot, our framework reduces the navigation task to reaching a nearby target while avoiding obstacles, enabling efficient learning with a low-dimensional state space and simple policy network. We analyze how the distinct force or torque signatures generated by walls at different relative positions influence the swimmer. Through comparative numerical experiments between the previous agent [53] and our augmented agent on a simple single-obstacle target pursuit task, we demonstrate that explicit force feedback enhances navigation efficiency, enabling faster target acquisition and obstacle avoidance. This finding contributes to advancing DRL strategies for robust underwater navigation in challenging, real-world applications such as cave exploration and infrastructure inspection.

The remainder of this paper is organized as follows. Section II presents the DRL framework. Section III analyzes wall effects on jellyfish-like swimming. Section IV evaluates navigation performance in environments with obstacles. Conclusions and future directions are summarized in Sec. V.

II. DEEP REINFORCEMENT LEARNING

The overall workflow is sketched in Fig. 1, which is extended from that in Ref. [53] by incorporating force and torque feedback. Multiple simulations were run to collect data of jellyfish-like swimming in Fig. 1(a). The dataset was then used for offline training of DRL agent in Fig. 1(g). The trained agent is employed for navigation in unknown environment with obstacles in Fig. 1(h) with pathfinding process illustrated in Fig. 1(f). Figures 1(b), 1(c), and 1(g) show the jellyfish's action space, state, and decision-making process, which are later detailed in Secs. II C and II D. Figures 1(d) and 1(e) illustrate the wall effect and how the agent exploits it and finishes the task in short time. The code and dataset of our study are available online [55].

A. Data preparation

The 2D flow data for the DRL of jellyfish-like swimming are obtained from numerical simulations. The immersed boundary method [56,57] is used to treat the fluid-solid coupling at the moving boundary. A unit density, incompressible flow is governed by the Navier-Stokes equations

$$\frac{\partial \boldsymbol{u}}{\partial t} + \boldsymbol{u} \cdot \nabla \boldsymbol{u} = -\nabla p + \nu \nabla^2 \boldsymbol{u} + \boldsymbol{f}, \tag{1}$$

$$\nabla \cdot \boldsymbol{u} = 0, \tag{2}$$

where $\boldsymbol{u}$, p, ν, and $\boldsymbol{f}$ denote the velocity, pressure, kinematic viscosity, and body force exerted by a jellyfish-like swimmer.

The immersed boundary is represented by Lagrangian markers. A regularized delta function δ_h [56] is employed to interpolate and spread $\boldsymbol{f}$ between Eulerian and Lagrangian points, whose coordinates are denoted by $\boldsymbol{x}$ and $\boldsymbol{X}$, respectively. The Eulerian force in Eq. (1) is calculated by

$$\boldsymbol{f}(\boldsymbol{x}) = \int_S \boldsymbol{F}(\boldsymbol{X}) \delta_h(\boldsymbol{x} - \boldsymbol{X}) d\boldsymbol{X}, \tag{3}$$

where $\boldsymbol{F}$ denotes the Lagrangian force at $\boldsymbol{X}$, and S the domain of the immersed boundary. The nonslip condition is satisfied by exerting $\boldsymbol{F}$ on the immersed boundary. The velocity on the immersed

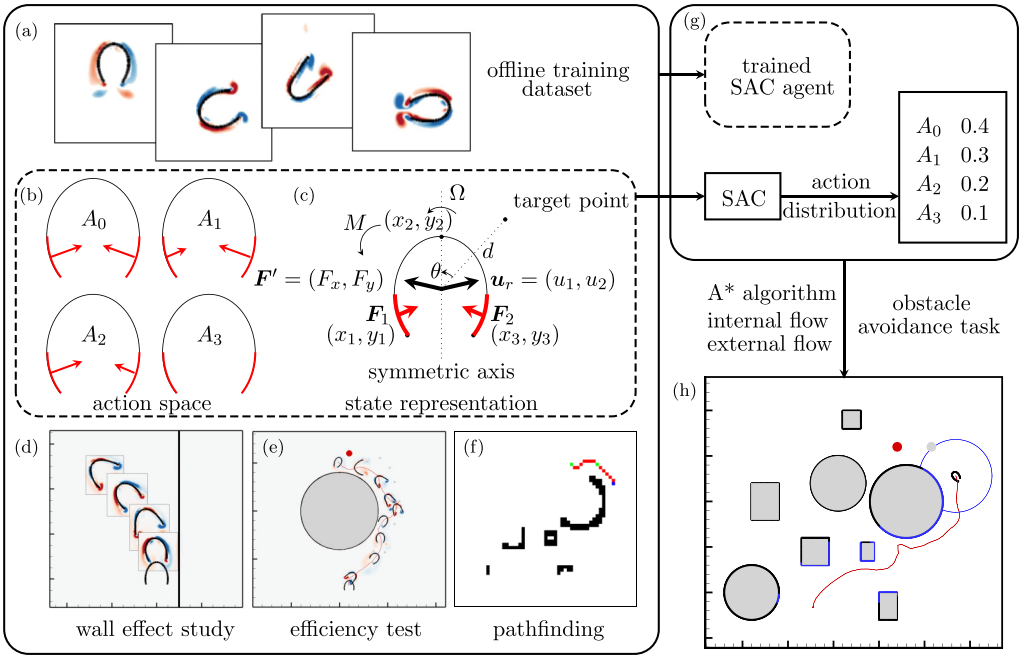


FIG. 1. Diagram for the overall workflow. (a) Data obtained from multiple simulations are used for offline training. (b) Action space with four actions A_i, $i = 0, 1, 2, 3$, representing typical jellyfish actions (from left to right): symmetric forces on the two sides, larger force on the right side, larger force on the left side, and no force. (c) Geometry and state of the jellyfish-like swimmer. The red parts indicate where the forces are applied. Test case studies: (d) the swimmer choosing the moving forward action deviates with the existence of a side wall; (e) the swimmer with SAC agent on a simple obstacle avoidance task; and (f) a pathfinding algorithm is performed in descretized domain based on detected boundaries. (g) The SAC module receives the state vector and outputs a probability distribution for the actions. The action is chosen with this distribution. (h) The swimmer senses the environment and uses the detected boundaries (in blue) to find a path in a map like (f). A pilot point is then calculated to guide the swimmer toward the target.

boundary satisfies

$$\int_{\mathcal{D}} \boldsymbol{u}(\boldsymbol{x})\delta_h(\boldsymbol{x} - \boldsymbol{X})d\boldsymbol{x} = \boldsymbol{U}_b(\boldsymbol{X}), \tag{4}$$

where $\mathcal{D}$ denotes the entire fluid domain and $\boldsymbol{U}_b$ the velocity at Lagrangian points.

The simulations were conducted using the code IBAMR [58], which is a distributed-memory parallel implementation of the immersed boundary method with adaptive mesh refinement for the Cartesian grid. The total number of grid points is in the order of 10^5, excepted that a few cases with higher Reynolds number used double of the resolution to ensure convergence. We have conducted convergence tests by doubling the grid resolution to ensure that the trajectory and quantities such as force and torque converge (see Appendix D).

We use the same data collecting and processing method as described in [53]. The training dataset was obtained from approximately 100 independent numerical simulations of the jellyfish-like swimmer in a square domain of size 32×32 (nondimensionalized by the swimmer diameter d_0) with periodic boundary conditions. The domain was discretized using adaptive mesh refinement, resulting in typical grid resolutions of 10^5 points. Each simulation lasted multiple swimming periods, during which the swimmer executed random actions uniformly sampled from the four discrete actions to encourage exploration of the state space. To ensure that the agent

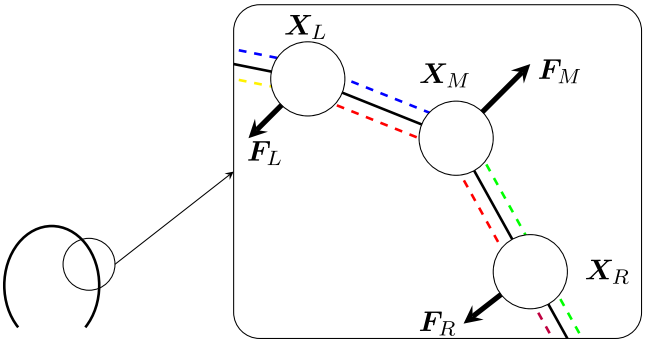


FIG. 2. The flexible swimmer consists of 159 Lagrangian points connected by springs and beams. Black solid lines and dashed colored lines represent springs and beams, respectively. The right panel details the three-point beam model, where $\boldsymbol{X}_L, \boldsymbol{X}_M, \boldsymbol{X}_R$ and $\boldsymbol{F}_L, \boldsymbol{F}_M, \boldsymbol{F}_R$ denote the positions and Lagrangian forces of the left, middle, and right points.

encounters vortex-wall interactions during training, the dataset includes both wall-free cases and cases with simple straight-line segment walls placed at various positions and orientations within the domain. The resulting state-action-reward transitions were stored and used as the replay buffer for offline SAC training. The dataset and the code used to generate it are publicly available at the repository [55].

In the present simulations, obstacles are represented as fixed Lagrangian marker points. Each marker is tethered to its initial position via a stiff virtual spring. The no-slip condition enforced at these markers prevents fluid penetration and, consequently, prevents the swimmer from crossing the obstacle boundary. Thus, blockage is naturally handled by the fluid-structure interaction solver without explicit collision detection. The jellyfish-like swimmer is modeled as a flexible body composed of Lagrangian points connected by springs and elastic beams.

The jellyfish-like swimmer is modeled as a flexible body composed of Lagrangian points connected by springs and elastic beams. Each adjacent pair of points is linked by a linear spring, and each triplet of neighboring points is connected by a beam that resists bending. The structure is sketched in Fig. 2.

The elastic force density is

$$\boldsymbol{F}_E(q,t) = \frac{\partial}{\partial q}\left(k_s\left|\frac{\partial \boldsymbol{X}(q,t)}{\partial q}\right| - 1\right)\boldsymbol{\tau}(q,t) - k_b\frac{\partial^4}{\partial q^4}[\boldsymbol{X}(q,t) - \boldsymbol{X}_b(q)], \tag{5}$$

where $\boldsymbol{X}(q,t)$ is the coordinate of Lagrangian material point q, $\boldsymbol{\tau} = (\partial \boldsymbol{X}/\partial q)/|\partial \boldsymbol{X}/\partial q|$ is the unit tangent vector, k_s and k_b are the stretching and bending stiffnesses, and $\boldsymbol{X}_b$ characterizes the resting shape.

For the beam connecting three neighboring points (left L, middle M, right R), a torsional spring formulation [59] is adopted. This replaces the linear beam used in earlier work [42] and is essential for enabling the swimmer to turn freely. The discrete restoring forces are

$$\begin{aligned}
\boldsymbol{F}_L &= k_b\,\Delta\vartheta\,(\boldsymbol{X}_M - \boldsymbol{X}_R)\times\boldsymbol{e}_z,\\
\boldsymbol{F}_M &= k_b\,\Delta\vartheta\,(\boldsymbol{X}_R - \boldsymbol{X}_L)\times\boldsymbol{e}_z,\\
\boldsymbol{F}_R &= k_b\,\Delta\vartheta\,(\boldsymbol{X}_L - \boldsymbol{X}_M)\times\boldsymbol{e}_z,
\end{aligned} \tag{6}$$

where $\boldsymbol{e}_z$ is the unit vector normal to the plane. The angular deviation $\Delta\vartheta$ measures the change in the angle between $\boldsymbol{X}_R - \boldsymbol{X}_M$ and $\boldsymbol{X}_M - \boldsymbol{X}_L$ relative to the resting configuration:

$$\Delta\vartheta = \boldsymbol{e}_z\cdot((\boldsymbol{X}_R - \boldsymbol{X}_M)\times(\boldsymbol{X}_M - \boldsymbol{X}_L) - (\boldsymbol{X}_{R0} - \boldsymbol{X}_{M0})\times(\boldsymbol{X}_{M0} - \boldsymbol{X}_{L0})), \tag{7}$$

with $\boldsymbol{X}_{L0}, \boldsymbol{X}_{M0}, \boldsymbol{X}_{R0}$ denoting the resting positions of the three points. Substituting the torsional spring forces into the total torque about the mass center yields $\boldsymbol{T} = 0$, confirming that this formulation produces no spurious internal torque under rigid-body rotation.

The resting geometry is an elliptical segment with semimajor axis 0.75 and semiminor axis 0.5 (nondimensionalized by the swimmer diameter d_0). The propulsive forces are applied along a small segment at each tip indicated in red in Fig. 1(c). Applying sinusoidal forces at the two tips induces bending and deformation. When the forces are removed, the body passively relaxes back to its resting shape due to elastic restoring forces. The fluid-structure interaction is fully two-way coupled, where the deforming swimmer exerts forces on the fluid via the immersed boundary method, and the fluid motion in turn advects and deforms the swimmer.

B. Soft actor-critic method

Soft actor-critic (SAC) [60] is a model-free, off-policy actor-critic reinforcement learning algorithm. It is designed to achieve a balance between exploration and exploitation by incorporating the principles of maximum entropy into reinforcement learning. This method not only maximizes the expected cumulative rewards but also encourages the policy to explore a wide range of actions, leading to more robust and stable learning. SAC extends traditional reinforcement learning objectives by adding an entropy term to the reward function. The modified objective aims to maximize the expected cumulative reward over a policy π as

$$J(\pi) = \mathbb{E}_{\pi}\left(\sum_{t=0}^{N_t} r_t + \alpha \mathcal{H}[\pi(\cdot|s_t)]\right), \tag{8}$$

where r_t is the reward received at time step t, $\pi(\cdot|s_t)$ the policy, which is the probability distribution of choosing each action at state s_t; $\mathcal{H}[\pi(\cdot|s_t)]$ represents the entropy of the policy, and temperature α controls the trade-off between maximizing rewards and maintaining high entropy. High entropy promotes exploration by encouraging diverse action sampling.

Unlike on-policy algorithms such as proximal policy optimization [61], SAC can be off-policy, meaning it can reuse past experiences stored in a replay buffer without interacting with the environment constantly. This makes SAC more sample-efficient and suitable for tasks where data collection is expensive or time-consuming. It is important to clarify that SAC is inherently an off-policy algorithm, not an offline one. The "offline" here describes our training setup, where the agent is trained exclusively on a precollected dataset without further environment interaction. This choice is motivated by the prohibitive computational cost of online reinforcement learning in CFD environments. Online training on a comparable task [60] typically requires on the order of one million steps. For our CFD environment, each simulation step takes approximately 60 seconds of wall-clock time, leading to an estimated runtime of roughly 16 667 hours, which is computationally expensive. Offline learning circumvents this issue by decoupling data collection from policy updates. We first generate a diverse dataset using random actions, apply data augmentation [53] to enlarge the sample size, and then train the SAC agent offline on this fixed dataset.

C. Case setup

The jellyfish-like locomotion is enabled by applying a pair of sinusoidal forces to the swimmer's tips (marked in red), as illustrated in Fig. 1(c). The force density (force per unit length)

$$\boldsymbol{F}_i = F_i \sin(2\pi f t)\boldsymbol{\tau}_i, \quad i = 1, 2, \tag{9}$$

acting on the left and right halves of the swimmer are denoted as $\boldsymbol{F}_1$ and $\boldsymbol{F}_2$, respectively, with $F_i = |\boldsymbol{F}_i|$ and the unit tangent vector $\boldsymbol{\tau}_i$ on the swimmer's tips that point inward. Note that F_1 and F_2 in Eq. (9) can be different, while they have the same frequency f and zero initial phase.

We divide a period $t = 0 \sim T$ of the sinusoidal force pair (F_1, F_2) applied to the swimmer into four quarters, with $T = 1/f$. The force pair is nondimensionalized by the reference force

TABLE I. Parameters in the simulation for jellyfish-like swimming.

Parameter	Symbol	Value
Swimmer diameter	d_0	0.1
Domain length	W	1.6
Frequency	f	0.5
Period	$T = 1/f$	2
Kinematic viscosity	ν	5×10^{-5}
Characteristic length	$l = d_0$	0.1
Characteristic velocity	$U = fl$	0.05
Reynolds number	$\mathrm{Re} = Ul/\nu$	100
Discretization length	Δx	$\frac{d_0}{2}$
Time step size	Δt	$\frac{T}{4}$
Sensing range	$\mathcal{R}$	$4d_0$

density 1 N/m. During each quarter, a SAC agent is employed to process the current state and produce (F_1, F_2). In order to reduce the training complexity, we restrict the choices for (F_1, F_2) to (0.003,0.003) for symmetric force application (moving forward), (0.001,0.003) for larger force on right-side muscle (turning right), (0.003,0.001) for larger force on left-side muscle (turning left), and (0,0) for no force applied (drifting), labeled as actions A_i with $i = 0$, 1, 2, and 3, respectively.

In our validation tests, the obstacles are unknown to the swimmer. Instead, the swimmer has a sensing range of $\mathcal{R} = 4d_0$, where d_0 is the diameter of the swimmer. The boundary Lagrangian points of obstacles are known to the swimmer only when they are within this range and not blocked by other obstacles. This occlusion check is implemented using a ray-casting approach. For each Lagrangian marker point of an obstacle within the sensing radius $\mathcal{R}$, we test whether the line segment connecting the swimmer's center to that marker intersects any other obstacle marker. If an intersection is found, the marker is considered occluded and is excluded from the locally known obstacle map used for path planning. This mimics the effect of visual obstruction in a natural environment. A fixed target point defines the navigation objective. Throughout the maneuver, an auxiliary target point (referred to as "pilot" below) is used. Specifically, at each navigation step, the A* algorithm computes an optimal path from the swimmer's current position to the target, utilizing the locally known obstacle map (see Appendix C for details). Then, a pilot is selected along this computed path. Its distance from the swimmer is set approximately equal to the sensing range $\mathcal{R}$. This positioning leverages the swimmer's maximum perception capability to provide timely guidance while avoiding obstacles.

The parameters for the swimmer are listed in Table I. The four actions patterns are shown in Fig. 1(b), and the same action regulation in [53] is also adopted. The action regulation mechanism follows the burst-and-coast strategy described in [53]. Forces are applied only during the first and third quarters of each swimming period, except for the symmetric action A_0 and the drifting action A_3, which can extend into subsequent quarters following specific action sequences. This regulation prevents interference between vortices generated in consecutive strokes and mimics natural jellyfish locomotion. The data augmentation is performed by shifting the target position in each collected sample of state-action-reward-next-state tuple, thereby varying d and θ [shown in Fig. 1(c)] without additional CFD simulations. This approach balances the distribution of state samples and effectively enlarges the dataset, as detailed in [53].

D. Training

The state of the swimmer is characterized as a 15-dimensional vector

$$\boldsymbol{s} = (x_1, y_1, x_2, y_2, x_3, y_3, u_1, u_2, d, \theta, \Omega, F_x, F_y, M, n), \tag{10}$$

where (x_i, y_i), $i = 1, 2, 3$ denote the coordinates of the swimmer's left, middle, and right endpoints referenced to its mass center, $\boldsymbol{u}_r = (u_1, u_2)$ the velocity of the mass center relative to target, d the distance between the mass center and target, θ the angle between the swimmer's symmetric axis and the line segment connecting the mass center and target, Ω the angular velocity of the symmetric axis, (F_x, F_y) the force exerted on the swimmer, M the torque refrenced to the swimmer's mass center, and $n = 0, 1, 2, 3$ indicates the quarter of period T. The angular velocity Ω is calculated as the change in the orientation angle of the swimmer's symmetric axis between consecutive time steps, where the symmetric axis is defined as the line segment connecting the mass center and the midpoint of the swimmer [53]. This kinematic definition avoids the need for explicit moment of inertia and torque-to-acceleration integration. Compared with the state vector in Ref. [53], F_X, F_y, and M are newly added. These force and torque signatures are highly correlated with wall proximity and orientation, which will be discussed in Sec. III.

The actions A_i, $i = 0, 1, 2, 3$ illustrated in Fig. 1(b) correspond to symmetric, asymmetric, and no force applied to two halves of the swimmer. The reward function [53]

$$r(\boldsymbol{s}, a) = -\min(\theta^2, 3) + \mathcal{A}\,\mathrm{clip}(v, -0.1, 0.1) - \mathcal{B}d \tag{11}$$

is adopted, where v is the projection of mass center's velocity onto the line segment connecting the mass center and target point, and constants $\mathcal{A}$ and $\mathcal{B}$ are tuned during the training; the clip function $\mathrm{clip}(x, b, c)$ returns x if $b \leqslant x \leqslant c$, b if $x < b$, and c if $x > c$. The reward function in Eq. (11) is designed to encourage three key behaviors: alignment with the target, forward progress, and proximity. The first term penalizes angular deviation from the target direction, with a cap of three to prevent excessive penalty when the swimmer is facing completely away. The second term $\mathcal{A}\,\mathrm{clip}(v, -0.1, 0.1)$ rewards velocity projected toward the target, clipped to avoid overemphasis on speed. The third term $-\mathcal{B}d$ penalizes distance to the target. The constants $\mathcal{A} = 20$ and $\mathcal{B} = 10$ are tuned to balance the three terms, as detailed in [53].

The training dataset is collected from CFD simulations with random actions and data augmentation method [53]. Unlike online RL, where the agent learns through continuous interaction with the environment, offline training relies entirely on a fixed dataset. This introduces two limitations. First, the policy may perform poorly on states that are underrepresented in the dataset, a problem known as distribution shift. Second, the agent cannot correct erroneous decisions through interactive exploration. Data augmentation partially addresses the first issue by expanding the coverage of the state space (see Appendix A). The second remains an inherent trade-off of the offline approach. We also note that offline training avoids the risk of catastrophic policy updates during online RL that could waste expensive CFD simulations, and it enables extensive hyperparameter tuning without additional CFD cost. Thus, while offline RL is not inherently more cost-effective per simulation step than online RL run with parallel environments, decoupling data collection from policy updates makes the overall problem tractable by drastically reducing the number of required CFD simulations. More details on training can be found in Appendix B.

III. WALL EFFECTS ON JELLYFISH-LIKE SWIMMING

A. Vortex-wall interaction

The presence of a wall exerts a significant influence on the surrounding flow field of a swimmer. Specifically, the wall generates a reflection flow field which affects the swimmer's trajectory. For example, as shown in Fig. 3(a), the swimming trajectory of a forward-swimming swimmer is strongly deflected when a side wall is introduced. We conducted simulations for multiple cases with varying initial wall distances d_w between the swimmer's mass center and the side wall for different Reynolds numbers (Re). Figure 3(b) plots the deflection angle φ of the swimmer's symmetric axis over a duration of $t/T = 10$.

The result reveals a distinct and nonmonotonic influence of the wall proximity at different Re. At low and moderate Re, the deflection angle exhibits a predictable monotonic decay as the wall distance increases. This decay is markedly slower in the low-Re case, a consequence of the

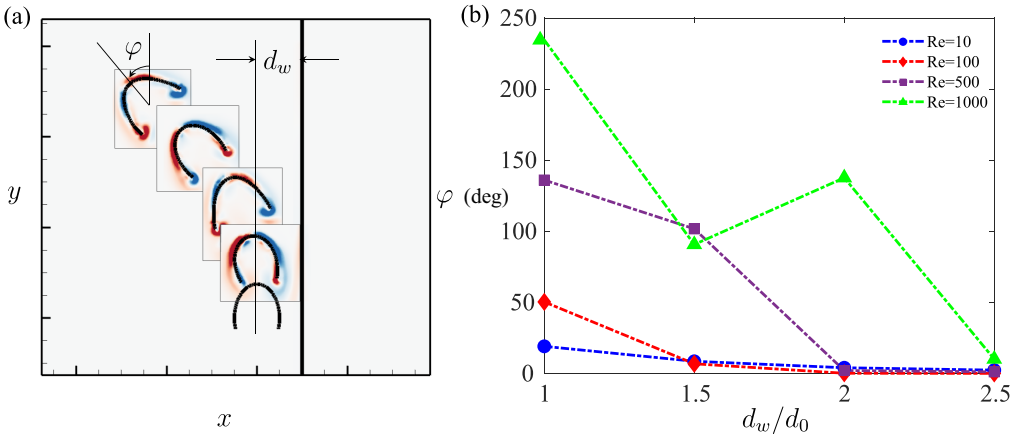


FIG. 3. (a) Trajectory (snapshots with gray edge) of the swimmer's forward motion with surrounding vorticity at $t/T = 0, 2.5, ..., 10$, where the action A_0 is always chosen, with a side wall, where φ and d_w are the deflection angle of the swimmer's symmetric axis and distance from the swimmer's mass center to wall, respectively. (b) Angle deviation of swimmer swimming forward over duration of $t/T = 10$ with different Reynolds numbers and initial distances to the side wall.

long-range nature of viscous interactions governed by the elliptic Stokes equations at the low-Re limit. In this regime, the no-slip condition at both the body and wall surfaces creates a globally coupled flow field where the viscous stress diffuses radially, ensuring that the wall's influence is felt strongly even at larger wall distances, resulting in a persistent deflection force. Figure 4 compares the trajectories of the swimmer at Re $= 10$ and 100 with $d_w/d_0 = 2.5$ over duration of $t/T = 10$. The swimmer at Re $= 10$ has a larger deflection angle and moves at a significantly lower speed than those at Re $= 100$ where it has almost zero deflection angle and minor displacement in the wall-normal direction.

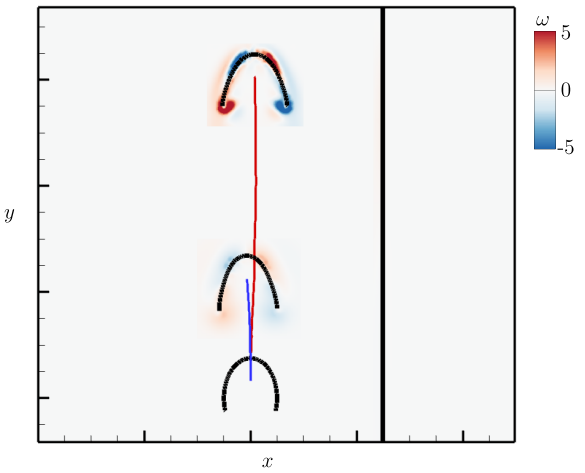


FIG. 4. Trajectory (snapshots at $t/T = 0$, 1, and 10) and vorticity field of the swimmer forward motion with a side wall at $d_w/d_0 = 2.5$ over duration of $t/T = 10$ for Re $= 10$ (blue dashed line) and 100 (red solid line).

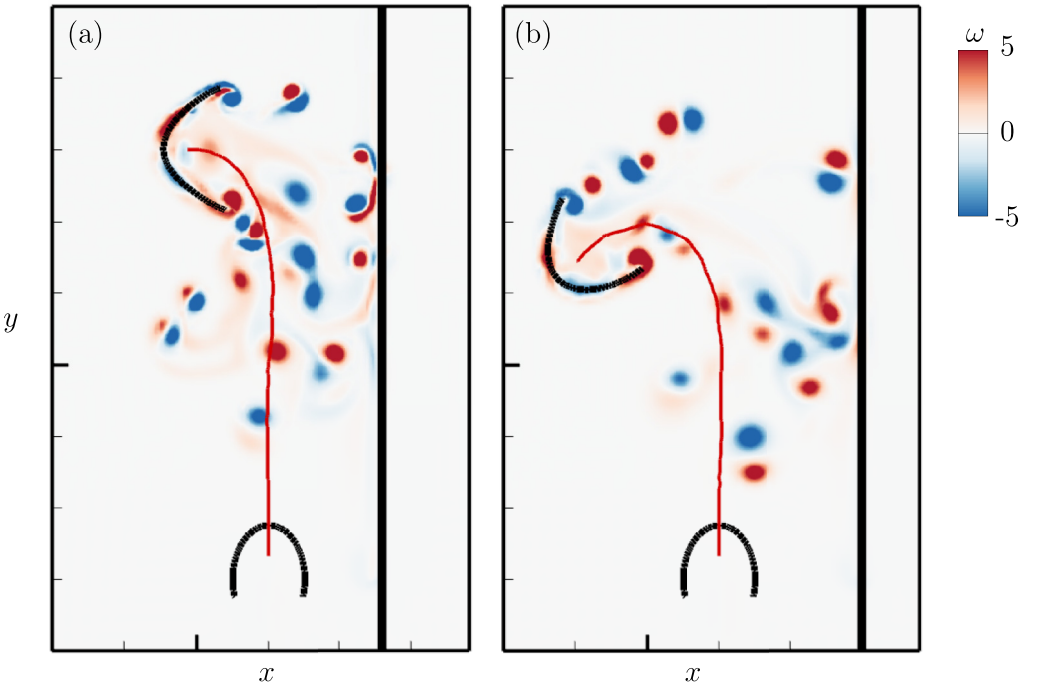


FIG. 5. Trajectory (red line) of the swimmer's forward motion over duration of $t/T = 10$ at Re $= 1000$, along with surrounding vorticity. (a) $d_w/d_0 = 1.5$. (b) $d_w/d_0 = 2$. The existence of the wall with different initial wall distances interacts differently with the shedding vortex from the swimmer and influences the deflection angle. Movie S1 [62] illustrates the comparision between the two scenarios.

At high Re, the trajectory becomes complex and exhibits no monotonic dependence on the initial wall distance. In Fig. 3(b) the deflection is significantly larger and even increases at intermediate wall distance $d_w/d_0 = 2$ before decaying for large initial wall distances. Figure 5 depicts the trajectories of the swimmer at Re $= 1000$ with $d_w/d_0 = 1.5$ and 2.0 (also see movie S1 in the Supplemental Material [62]).

The unpredictable trajectory of the swimmer is due to the interaction of wake vortices and the wall [35]. The swimmer can generate a series of vortices at moderate and higher Re. At a critical intermediate distance, the wall reflects and reorganizes the swimmer body's shed vortices. This interaction disrupts the stable, periodic vortex shedding pattern that would occur in an unbounded flow. The result is a feedback mechanism where the distorted vorticity field asymmetrically modifies the pressure distribution around the body, particularly its aft section, leading to an amplified and potentially unsteady lateral force. This interaction is weakened with the separation between the swimmer and the wall, as the deflection angle goes to zero at larger d_w/d_0.

Next, we examine the differences in the swimmer's force and torque patterns in the presence and absence of a wall. Figure 6 compares these quantities against the wall-free case. Since the orientation and surrounding flow is significantly different at later times, these quantities are only comparable at early times $t/T \leqslant 2$, where the the only difference is d_w. In Figs. 6(a) and 6(b), both the magnitude and direction of the force for different d_w deviate at $t/T \leqslant 2$. Figure 6(c) reveals that the torque differs substantially even at very early times $t/T \leqslant 0.5$, while Fig. 6(d) indicates that the displacement in the x direction remains nearly zero. Despite the distance $d_w/d_0 \geqslant 2$, a small but nonzero negative x displacement is observed, highlighting the complex vortex-wall interactions. These distinctions in force and torque demonstrate the influence of the vortex-wall interaction when the swimmer is near a wall.

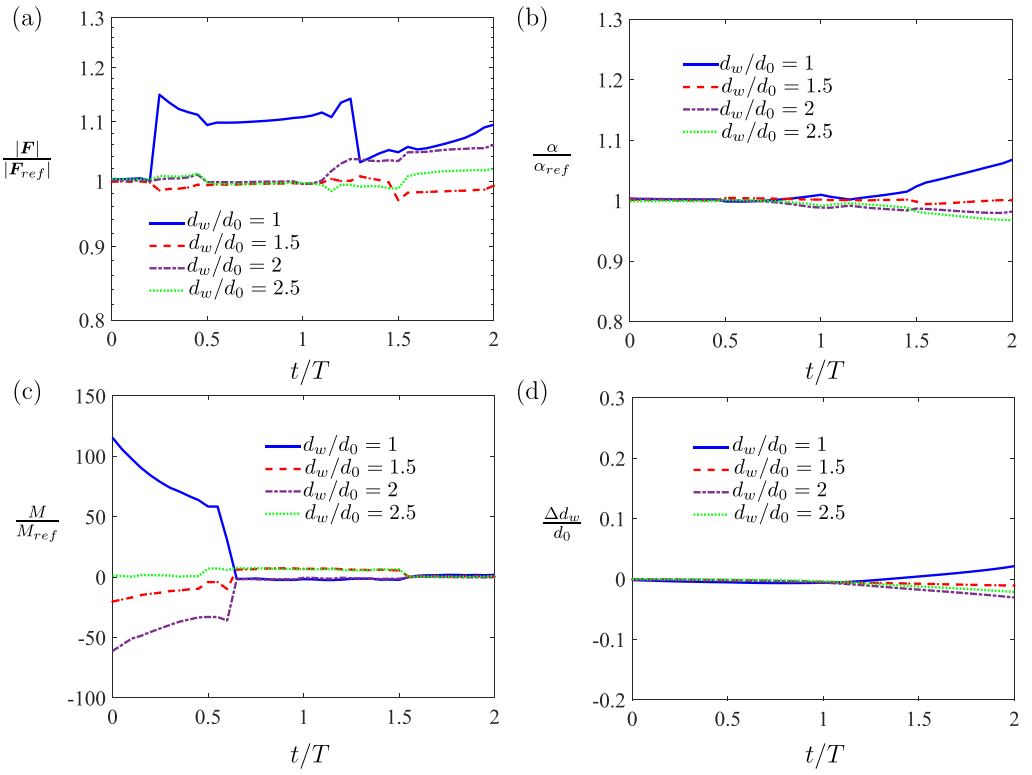


FIG. 6. Time evolution of (a) the swimmer's force magnitude, (b) force direction angle, and (c) torque, which are normalized by their wall-free values. (d) Corresponding displacement in the x direction. All simulations are conducted at Re $= 100$ with various d_w/d_0.

In particular, Fig. 6(c) indicates that the notable torque acting on the swimmer at Re $= 100$ deviates significantly from nearly zero torque in the wall-free scenario at the initial stage, regardless of the specific wall distance. Furthermore, Fig. 7 plots the initial torque at $t/T = 0.1$ as a function of d_w/d_0 for various Re, confirming that wall proximity induces substantial and nonmonotonic changes in torque. The strong dependence on both d_w and Re highlights the significance of torque. Since the torque determines the moving direction of the swimmer, it is essential to be perceived by an agent with measuring its hydrodynamic surroundings accurately. Additionally, we examined the effect of wall when it is put in front of the swimmer, and found that the presence of a front wall has minor influence on the propulsion of the swimmer (not shown).

B. Wall effects in obstacle avoidance

Based on our existing tracking strategy [53], we incorporate a pilot which updates every time step for the swimmer to track the actual specified target. To reach the target point and avoid obstacles, a pathfinding algorithm "A*" [63] is used to generate pilots. The environment is set up within the 2D computational domain, which is discretized with a grid spacing of $\Delta x = d_0/2$ for path planning purposes, and is effectively represented as a grid-based maze. In the immersed boundary method, solid walls are represented by closely spaced Lagrangian points. A grid cell is designated as an "obstacle" if it contains at least one Lagrangian point representing the obstacle boundary. This discretization is used by the A* algorithm, not by the CFD simulation.

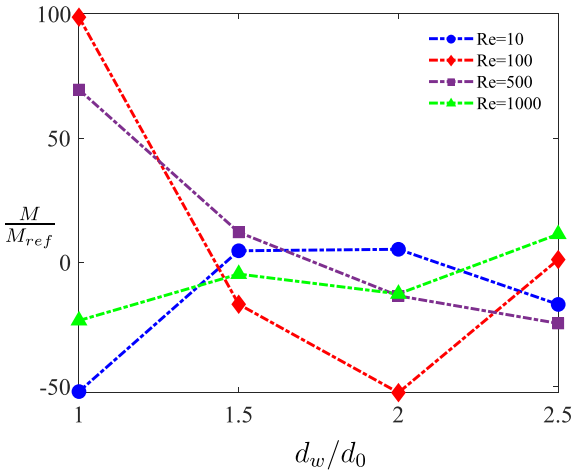


FIG. 7. The swimmer's torque at beginning stage normalized by the wall-free case for different d_w and Re.

Crucially, the swimmer possesses no prior knowledge of obstacle locations within the domain. Instead, it can detect obstacle Lagrangian points located within a specified sensing radius. To mimic realistic perception, obstacle points within this sensing range remain undetectable if they are visually blocked by other obstacles from the swimmer's viewpoint. Given this grid maze representation with locally sensed obstacle information, the A* algorithm (detailed in Appendix C) is employed for pathfinding. This algorithm generates an optimal path from the swimmer's current position to a designated target. Subsequently, a path-following approach is utilized to derive a pilot for the swimmer to track along this computed path. Note that the map detection and swimming are conducted together and the swimmer gradually detect its surroundings.

Figure 8(a) shows the trajectory of the swimmer in a simple obstacle avoidance tracking task. The swimmer manages to move along the edge of the cylinder obstacle and reach the target.As discussed for Fig. 3, the influence range of wall effect for current setup is around $2d_0$. Figure 8(c) shows that during nearly the whole process, the swimmer is within the influence range. Initially, the swimmer approaches the wall and experiences a repulsive force that pushes it backward (station 1). It then nearly makes contact with the wall and utilizes this repulsion to facilitate a rapid turn (station 2). Following the turn, the swimmer consistently selects action A_0 to propel itself forward. Throughout this phase, it counteracts the repulsive force from the wall to maintain a stable distance (stations 3–6). As it approaches the target, the swimmer selects action A_3 constantly to once again leverage wall interactions, executing a final turn to achieve the target position (stations 6 and 7).

Figure 8(d) details the swimmer utilizing the wall to execute a turn. Throughout this near-wall maneuver, action A_3 is consistently selected after the selection of A_1, allowing the swimmer to drift passively with the surrounding flow. The turn is accelerated by the vortex-wall interaction, where flow reflections impart momentum to propel the swimmer away from the boundary. Combined with the velocity produced by the action A_1 previously selected, this enables a rapid reorientation within a confined space.

For comparison, Fig. 8(b) shows the trajectory of a swimmer without force and torque sensing as agent inputs, performing the same task as in Fig. 8(a). Without sensing the surrounding flow's force (F_x, F_y) and torque M, this swimmer is pushed away from the obstacle by the reflected flow and fails to maintain proximity to the wall. It only reorients towards the target once sufficiently distant from the obstacle.

This phenomenon can be attributed to the incompleteness of the state representation when force and torque are omitted. Consider an agent whose state is represented by a 12D vector without mechanical feedback. In an obstacle-free environment, the transition from state $\boldsymbol{s}$ after taking action

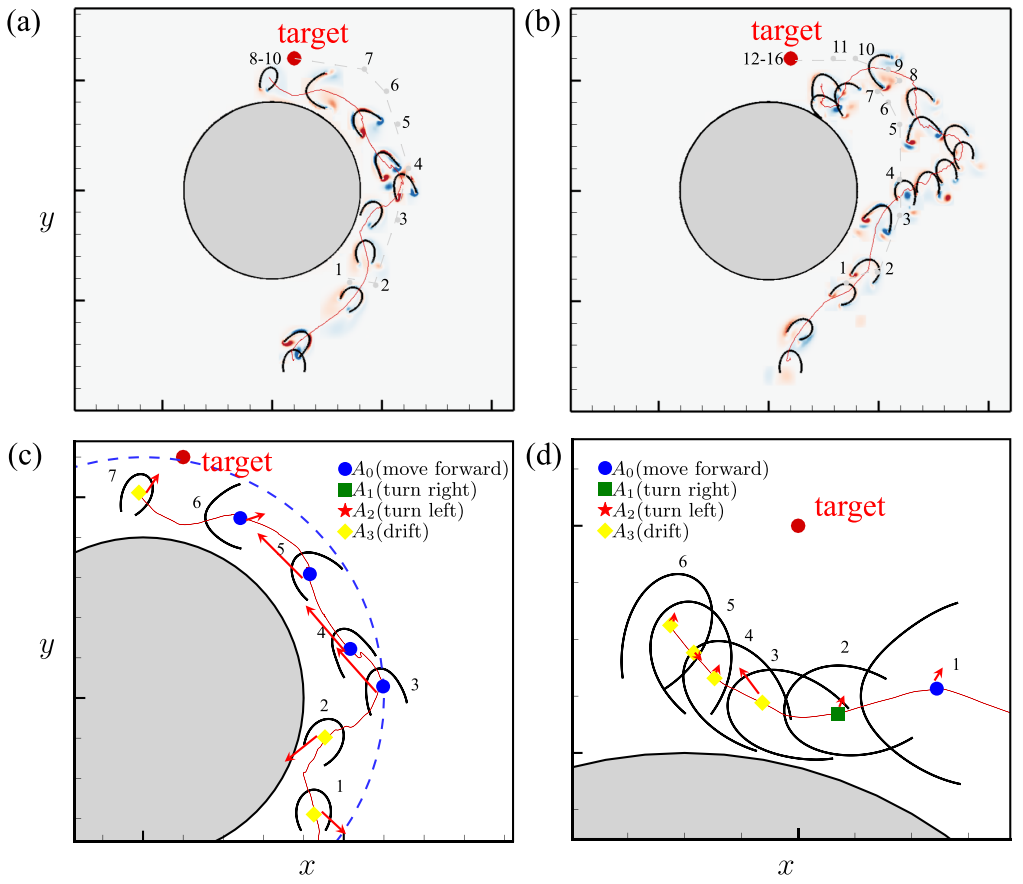


FIG. 8. Trajectory (red line) of the swimmer and surrounding vorticity field in a simple obstacle avoidance task with (a) current SAC agent at $t/T = 0, 5, ..., 45$, and (b) SAC agent without force and torque input at $t/T = 0, 5, ..., 75$. Gray dots and line with stations are pilots and their trajectory used to guide the swimmer. (c) Trajectory of the swimmer (red line) with current SAC agent and the force experienced (red arrow) and action choice over $t/T = 15, 20, ..., 45$ and (d) $t/T = 40, 41, ..., 45$. Blue dotted line represents the influence range of the obstacle.

A is deterministic, leading to a specific next state $\boldsymbol{s}_1$. However, when obstacles are present, the same state $\boldsymbol{s}$ (which lacks information about wall proximity) can result in different next states depending on the obstacle configuration, e.g., $\boldsymbol{s}_2$ instead of $\boldsymbol{s}_1$, because the hydrodynamic influence of the wall alters the swimmer's motion. Consequently, the Bellman optimality equation still holds, but the Q-function approximated on this incomplete state space cannot correctly capture the value of state-action pairs. By augmenting the state with force and torque feedback, we recover a more complete representation. In the framework of partially observable Markov decision processes, the augmented observation restores the Markov property, enabling the agent to learn an optimal policy via standard Bellman updates.

To assess the individual contributions of force and torque feedback, we trained two additional SAC agents with the same offline pipeline and dataset. The first includes only the force components (F_x, F_y) in its state vector, and the second includes only the torque M. These two agents were tested on the single-obstacle task of Fig. 8, and their trajectories are compared in Fig. 9. Both the agents outperform the 12D baseline, with completion times of $60T$ and $50T$ versus $75T$ for the

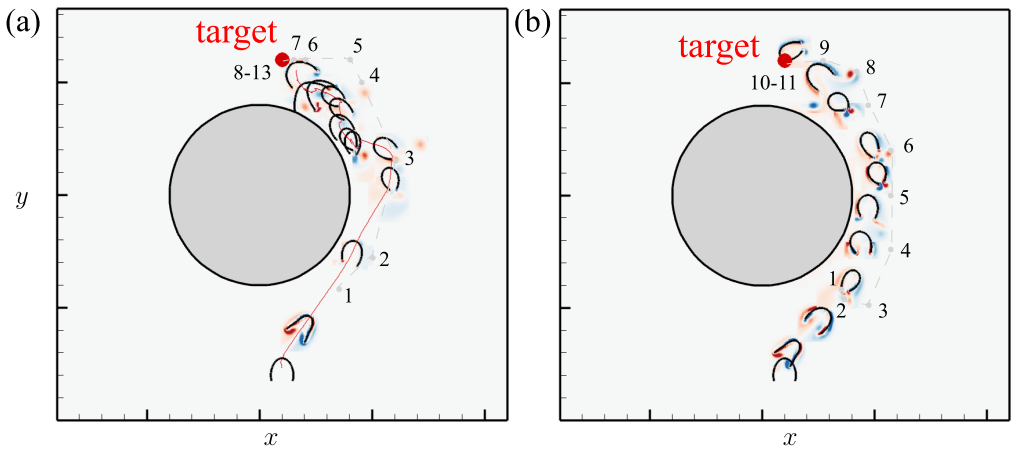


FIG. 9. Trajectory (red line) of the swimmer and surrounding vorticity field in a simple obstacle avoidance task with (a) force-only SAC agent at $t/T = 0, 5, ..., 60$, and (b) torque-only SAC agent at $t/T = 0, 5, ..., 50$. Gray dots and line with stations are pilots and their trajectory used to guide the swimmer. All agents are tested under identical conditions.

baseline. The force-only agent shows clear improvement over the 12D baseline during the initial wall-approach phase, indicating that force components help detect wall proximity. In the latter half of the trajectory, its performance degrades where it struggles to reach to target quickly near the wall. By contrast, the torque-only agent performs similarly to the 15D agent throughout the task. In the later near-wall segment, it maneuvers more flexibly than the force-only agent, maintaining close wall proximity and exploiting vortex-wall interactions for efficient reorientation. These results confirm that both mechanical signals contribute to navigation performance, and that torque plays a particularly critical role in sensing wall orientation during sustained near-wall locomotion.

Figure 10 compares action selections over time. The time interval is divided into two phases, 0–$22.5T$ and $22.5T$–$45T$, corresponding to the distinct stages of the trajectory in Figs. 8(a) and 8(b), which are the initial wall approach (stations 1–3) and the subsequent wall-following phase toward the target (stations 4–7). This separation highlights how action selection differs

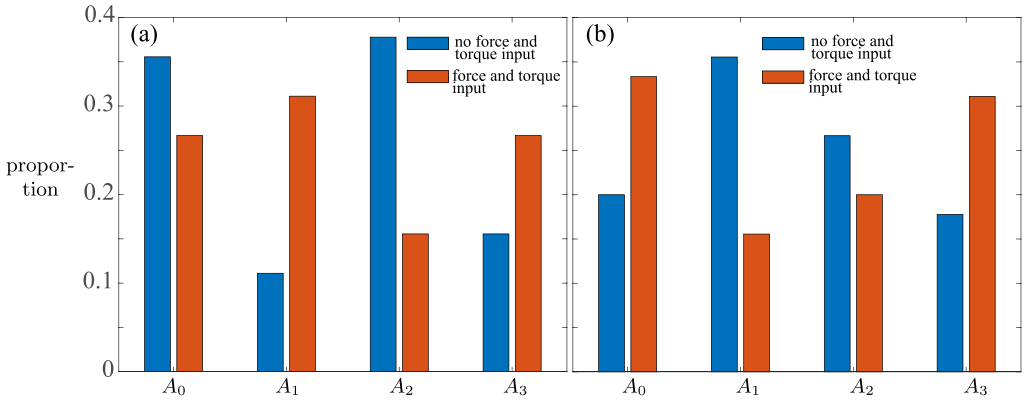


FIG. 10. Comparison of action-selection proportion histogram between models with or without force and torque input at (a) $t/T = 0$–22.5 and at (b) $t/T = 22.5$–45. The model with force and torque input selects more A_1 to turns more at the early stage and A_0 to stabilize its orientation.

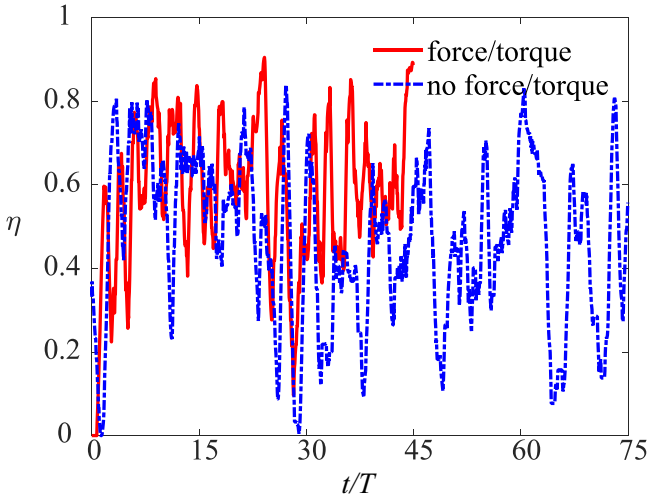


FIG. 11. Comparison of swimming efficiency of two agents. Current agent has a higher average of 0.58 than the baseline's 0.48.

between approaching the wall and exploiting it for navigation. During the initial $22.5T$ interval, the present model (SAC agent with force and torque input) predominantly selects A_1, inducing sustained rightward turning that positions the swimmer away from the obstacle. Subsequently, it reorients toward the obstacle, ultimately harnessing wall contact to rapidly shift direction. In the latter $22.5T$ phase, increased A_0 selection maintains obstacle clearance while mitigating repulsive wall-interaction forces. The maneuver concludes by exploiting wall-induced flow to align with the target direction. Conversely, the old model (SAC agent without force and torque input) [53] selects A_1 less frequently, favoring A_2 to pursue the target. During the second $22.5T$, wall-induced flow propels it away from the boundary, necessitating extended realignment to achieve target orientation. Note that the baseline agent without force and torque feedback was trained using the same dataset and the same SAC algorithm as the proposed agent, ensuring a fair comparison. While an ablation study considering force and torque separately could provide further insights into their individual contributions, it lies beyond the scope of the present work, which focuses on demonstrating the overall benefit of incorporating both mechanical signals.

Figure 11 shows the plot of swimming efficiency (η) of the two agents. The swimming efficiency is a modified form of the Froude efficiency [5], defined as

$$\eta = \frac{P_{\text{thrust}}}{P_{\text{thrust}} + \max(P_{\text{def}}, 0)}. \tag{12}$$

It measures the proportion of power used for propulsion other than deformation, where P_{thrust} and P_{def} represent the power output related to thrust generated by the body, and the power exerted in deforming the swimmer's body against fluid-induced forces, respectively. The two agents have average swimming efficiency of 0.58 and 0.48. Therefore the new agent has advantage over the baseline on both swimming efficiency (0.58 over 0.48) and completion time ($45T$ over $75T$).

Similarly, Figs. 12(a) and 12(b) show the trajectory comparison in a tracking task that requires a U turn. The agent with force and torque input has completion time $35T$ and swimming efficiency 0.54, where the baseline agent has $47T$ and 0.44. The current agent performs better by executing a faster and smaller U turn than the baseline, indicating the advantage of force and torque input.

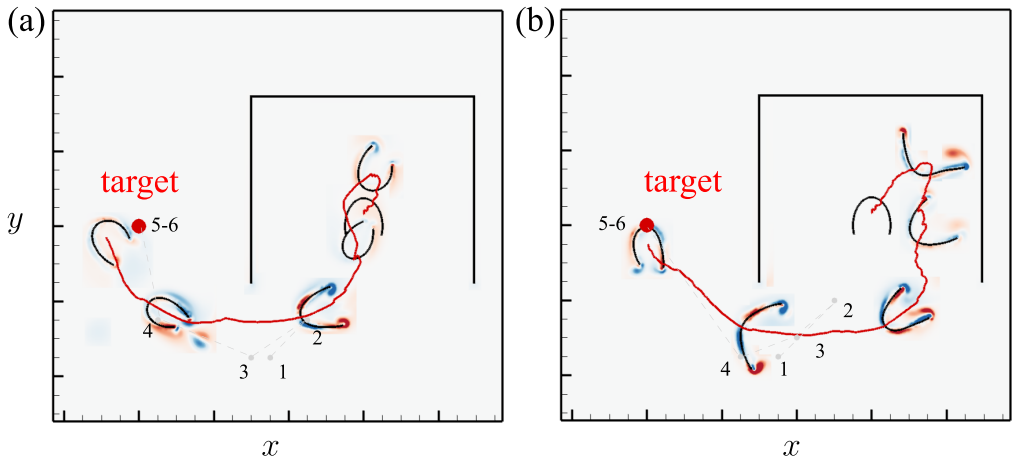


FIG. 12. Trajectory (red line) of the swimmer and surrounding vorticity field in a U-turn task with (a) current SAC agent at $t/T = 0, 7, ..., 35$, and (b) SAC agent without force and torque input at $t/T = 0, 9.5, ..., 47$. Gray dots and line with stations are pilots and their trajectory used to guide the swimmer.

IV. NAVIGATION IN ENVIRONMENT WITH OBSTACLES

Next we will illustrate the performance of DRL agent with force and torque input using tracking tasks in environments with complicated obstacles. The schematic of how the swimmer detects environment and derives pilot is shown in Fig. 13 (also see movie S2 [62]).

We will first assess the swimmer's locomotion and obstacle avoidance capabilities in an open flow domain containing obstacles (external flow scenario). Subsequently, to evaluate performance

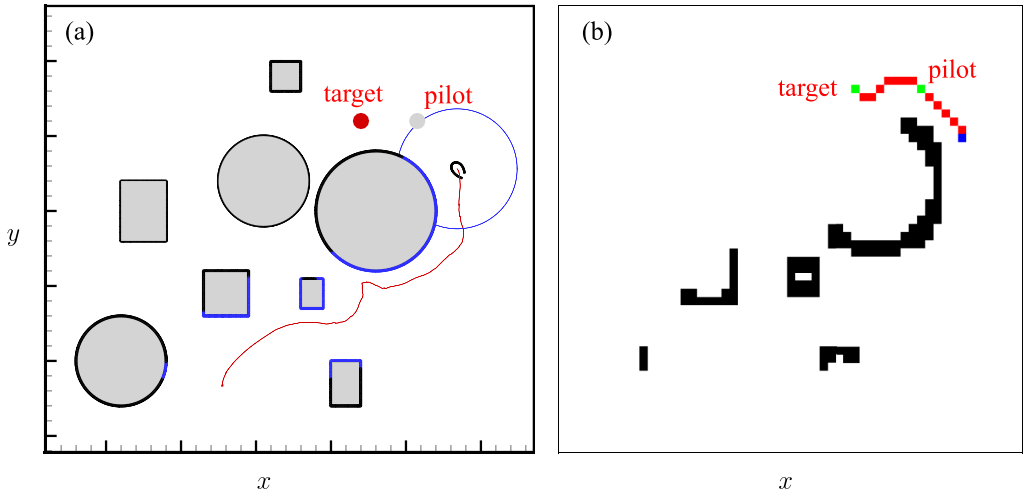


FIG. 13. (a) Trajectory of the swimmer (red line). The swimmer has a sensing range of $\mathcal{R} = 4d_0$ (blue circle). The boundaries sensed by the swimmer are updated in blue. (b) Based on the sensed boundaries (black dots), the domain is descretized, and A* algorithm is applied to generate path (red dots) from the swimmer (blue dot) to the target with a pilot (green dots). The pilot is chosen such that it is on the path and has an approximate distance of $\mathcal{R}$ from the swimmer. The tracking process is shown in movie S2 [62].

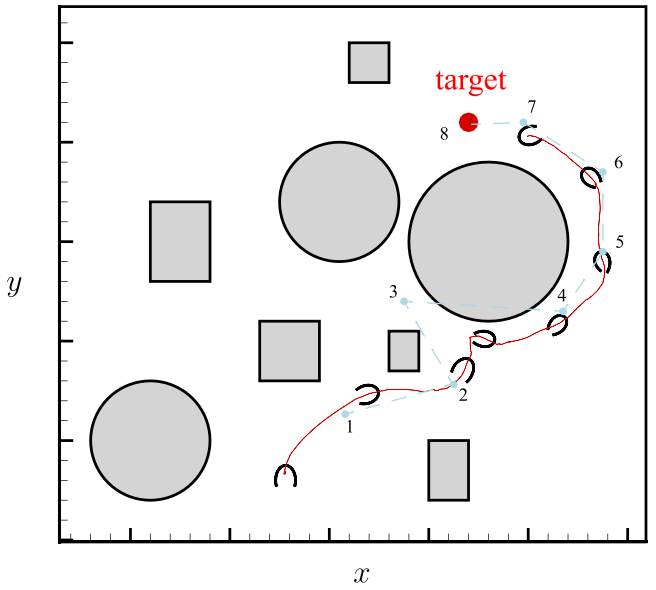


FIG. 14. Trajectory (red line) of swimmer at $t/T = 0, 9, \ldots, 63$. Light blue dots and line with stations are pilots and their trajectory used to guide the swimmer, respectively.

for real-world applications such as cave diving exploration, the swimmer navigates towards a predefined target with navigating complex, enclosed, mazelike domains (internal flow scenario).

A. Navigation in external flow

The swimmer navigates an open environment containing unknown, randomly distributed, and irregular obstacles. Figure 14 depicts its trajectory. It initially advance northeast until encountering the first obstacle, where hydrodynamic repulsion forces induce backward displacement. Subsequently, it circumvents the obstacle and resumes forward propulsion. At $t/T = 27$, collision with a second obstacle occurs, and the swimmer harnesses this interaction to rapidly execute a rightward turn via wall-induced vorticity. It then maintains boundary-proximal locomotion until successfully reaching the target.

The path-planning sequence is illustrated in Fig. 15. Initially, the A* algorithm generates a path through the gap between two detected obstacles. However, due to inertia, the swimmer deviates to the right side of these obstacles during $t/T = 9$–18, triggering the first path replanning. As shown by the pilot at station 3 of Fig. 14, the path is initially adjusted to pass between the square and cylindrical obstacles. A second replanning is then activated, shifting the path further rightward. This newly computed path proves more effective by better balancing the dual objectives of trajectory length minimization and obstacle avoidance. Following the replanning, the pilot proceeds along the cylindrical obstacle and successfully guides the swimmer toward the actual target.

B. Exploration in internal flow

In real-world underwater exploration or rescue missions, targets are often unknown in advance. Therefore, the swimmer must conduct a comprehensive exploration of the environment to detect potential targets of interest. We model the geometry of a real cave [64] and define the task as fully exploring the cave and returning to the starting position. The swimmer has a sensing range of $\mathcal{R} = 4d_0$, which is similar to the box jellyfish *Tripedalia cystophora* in natural environment [65]. Boundaries within this range that are not obstructed by other boundaries are considered explored.

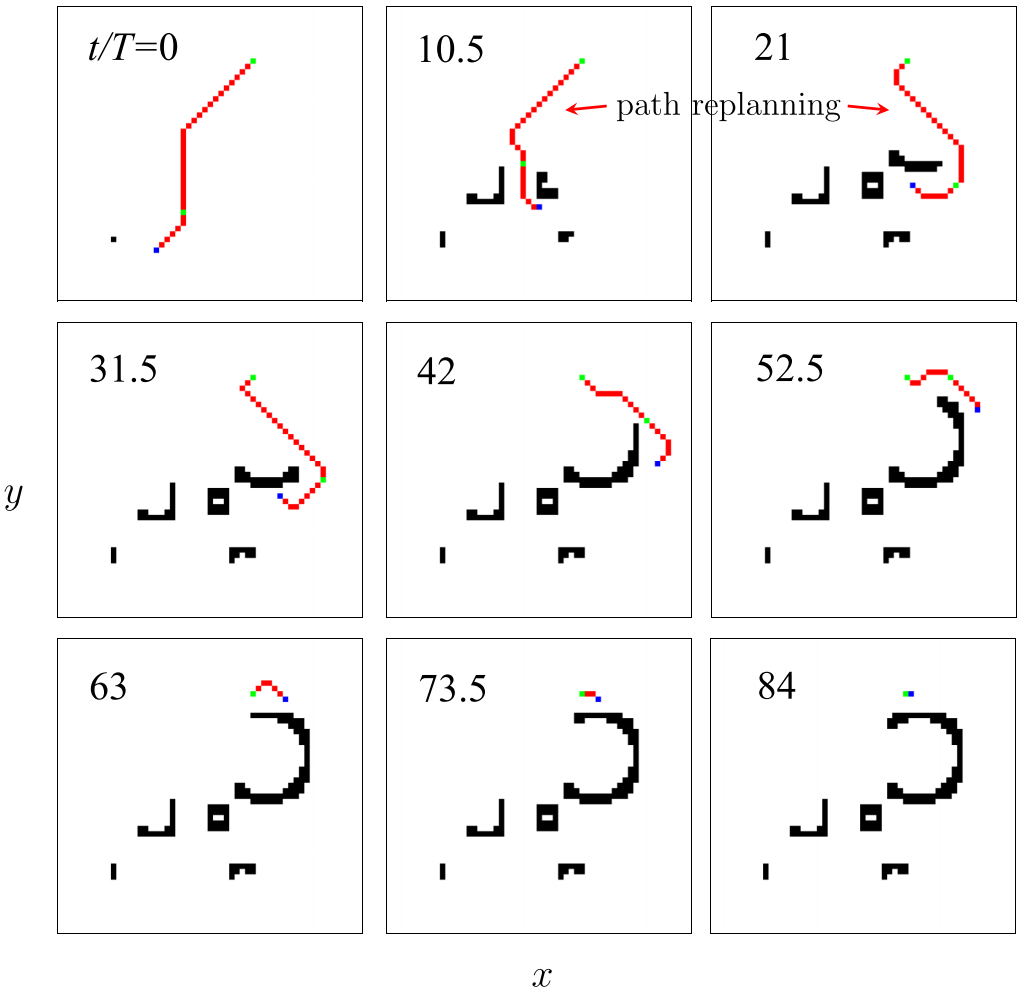


FIG. 15. Maze map constructed in real time with position of the swimmer (blue dot), planned path (red dots), pilot and target (both in green dots) at different t/T (marked at the left upper of each panel).

The cave modeled with a closed boundary is considered fully explored once its entire boundary has been covered.

Figure 16 illustrates the trajectory of the swimmer during the exploration process (also see movie S3 [62]). The swimmer initially advances forward to conduct exploration of the environment. When it encounters a fork in the path in Fig. 16(a), it opts to proceed along the left branch first. After reaching the end of this branch, the swimmer backtracks and then investigates the alternative fork. Once this section has been thoroughly explored, the entire cave is fully mapped, prompting the swimmer to begin its return journey to the starting point. During the return navigation through the narrow passage leading to the origin, the swimmer's trajectory becomes noticeably sinuous due to the wall effect. This effect induces complex hydrodynamic interactions, causing deviations in the intended path. Despite being pushed backward on two occasions by the surrounding flow, the swimmer demonstrates robust navigational capability and succeeds in returning to the origin.

We clarify that the exploration behavior shown in Fig. 16 is determined by an external path planning algorithm, rather than being learned by the RL agent. The RL agent is trained only to navigate toward a given local target (the pilot) while avoiding obstacles, using force and torque

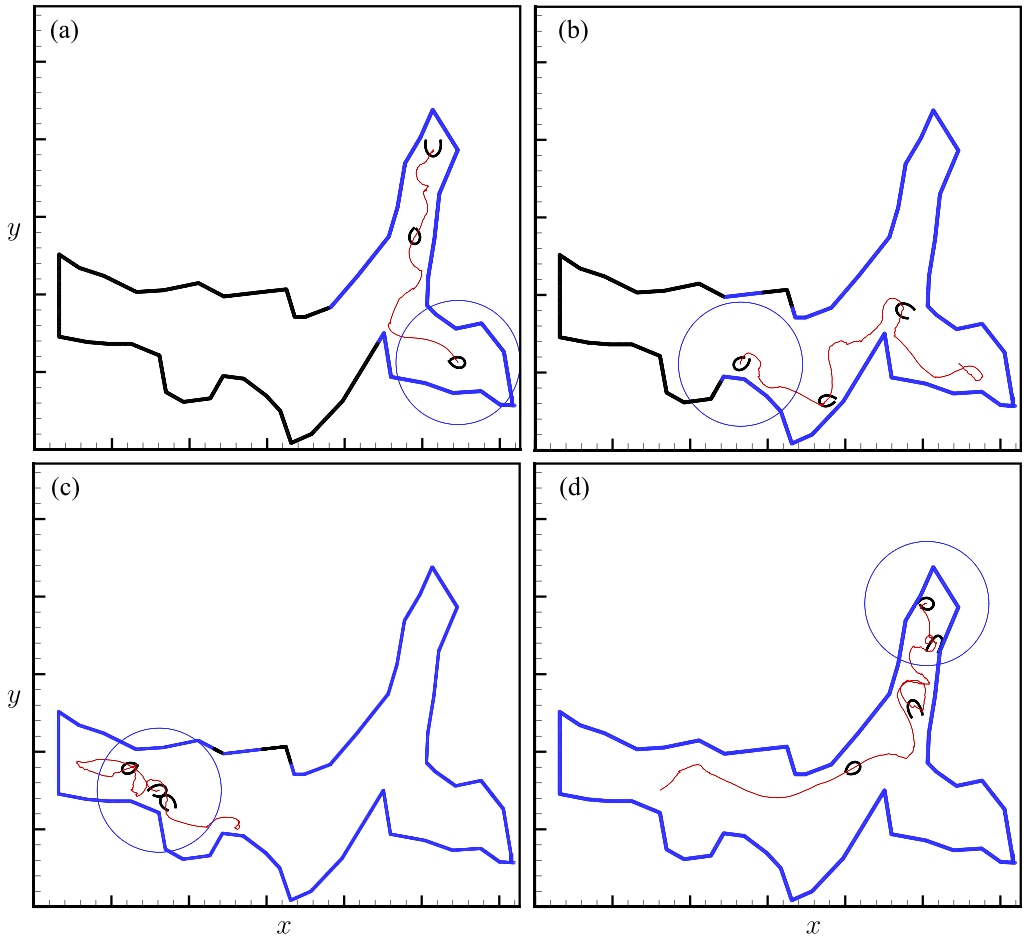


FIG. 16. Trajectory of swimmer (in red line) at (a) $t/T = 0, 25, 50$. (b) $t/T = 75, 100, 125$. (c) $t/T = 150, 175, 200$. (d) $t/T = 225, 250, 275, 300$. The sensing range is represented by the blue circle. Explored boundary is updated in blue. Movie S3 [62] illustrates the exploring process.

feedback to exploit wall effects. The high-level exploration strategy is rule-based: at each step, the swimmer detects unexplored boundary regions within its sensing range, and the target is set to guide it toward the nearest unexplored frontier (e.g., the entrance of a new branch). The pilot is then computed using the A* algorithm. Once the entire cave boundary has been covered, the target is switched to the starting point to guide the swimmer back. This decomposition allows the RL agent to focus on low-level control in complex near-wall flows, while high-level exploration is handled by a deterministic planner.

As shown in Fig. 17, the swimmer performs turning maneuvers analogous to the strategy in Fig. 8(d). Figures 17(a) and 17(b) capture two such turn-back events during the mission. This behavior indicates that the swimmer is endowed with the capability to detect frontal obstacles through real-time force and torque sensing, enabling proactive turn initiation. Consequently, it not only avoids collisions but also strategically exploits the wall reflection effect to execute efficient turns.

The path planning strategy is depicted in Fig. 18. This strategy involves continuously reassigning the target to the nearest open end within the explored environment. In the presented scenario, because the path is straightforward and the target remains in close vicinity to the swimmer, the pilot is set to be identical to the primary target.

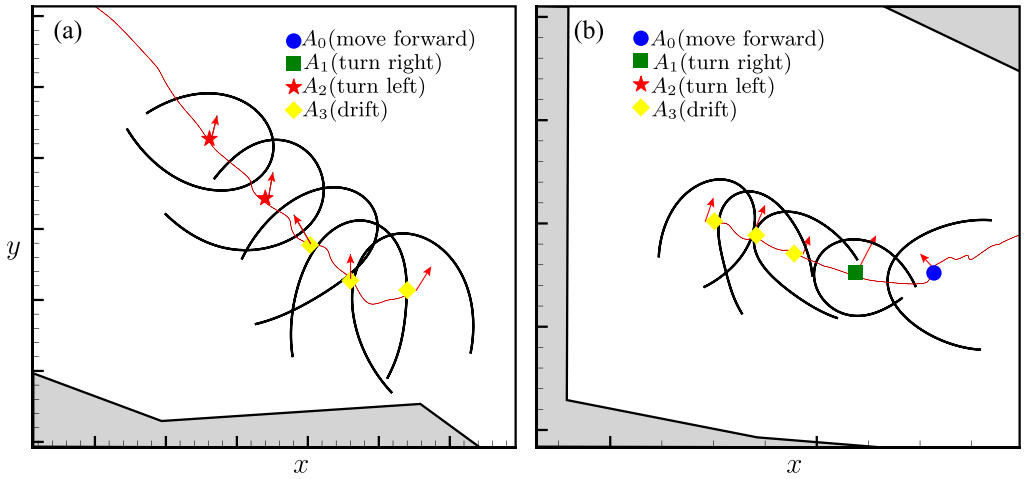


FIG. 17. (a) Trajectory of the swimmer (red line) with current SAC agent and the force experienced (red arrow) and action choice over $t/T = 50, 51.25, ..., 55$ and (b) $t/T = 180, 181.25, ..., 185$ in cave exploration task. They correspond to two turnings at the end of the two branches in Figs. 16(a) and 16(c).

V. CONCLUSION

We present a DRL framework for autonomous navigation of a jellyfish-like swimmer in complex, obstacle-laden fluid environments, specifically addressing the significant influence of vortex-wall interactions on swimming near the wall. By augmenting the agent's state representation to explicitly include the real-time forces and torque experienced by the swimmer, we demonstrate how DRL agents perceive and respond to wall boundaries. Our analysis reveals that these mechanical feedback signals provide necessary information, which is absent in the previous model [53] with kinematic and geometric state spaces (e.g., position and velocity).

Especially, vortex-wall interactions induce significant and often nonmonotonic variations in the forces and torque acting on the swimmer at moderate to high Reynolds numbers. These interactions arise from the reflection and reorganization of the swimmer's shed vortices by nearby walls, leading to asymmetric pressure distributions and unsteady lateral forces. Such hydrodynamic phenomena are captured in real time through the force and torque signals, which serve as a highly sensitive directional proximity sensor. For instance, strong frontal drag indicates a head-on obstacle, while lateral forces and torque reveal the presence and orientation of side walls. These mechanical cues allow the agent to detect complex near-field flow effects, such as pressure gradients and vortex shedding, that are inaccessible through geometric or kinematic states alone.

This direct physical feedback improves the agent's decision making. Compared to previous agent lacking force and torque input, our augmented agent exhibits anticipatory and targeted maneuvers, initiating smoother turns earlier and exploiting wall-induced forces (e.g., leveraging lateral drag for efficient reorientation) rather than resorting to reactive, oscillatory collision-avoidance behaviors. This leads to enhanced performance in both simple target pursuit with obstacles and complex navigation scenarios, characterized by faster task completion and higher swimming efficiency (33% faster and 10% higher for the single obstacle task).

Our findings underscore the critical role of embodied mechanical interaction for intelligent control in fluid-immersed systems. The success of force and torque feedback mirrors principles in robotic tactile perception, where direct contact forces guide dexterous manipulation. This work validates the feasibility of using DRL-controlled bio-inspired swimmers for high-risk applications like underwater cave exploration. More broadly, it establishes force and torque feedback as a vital

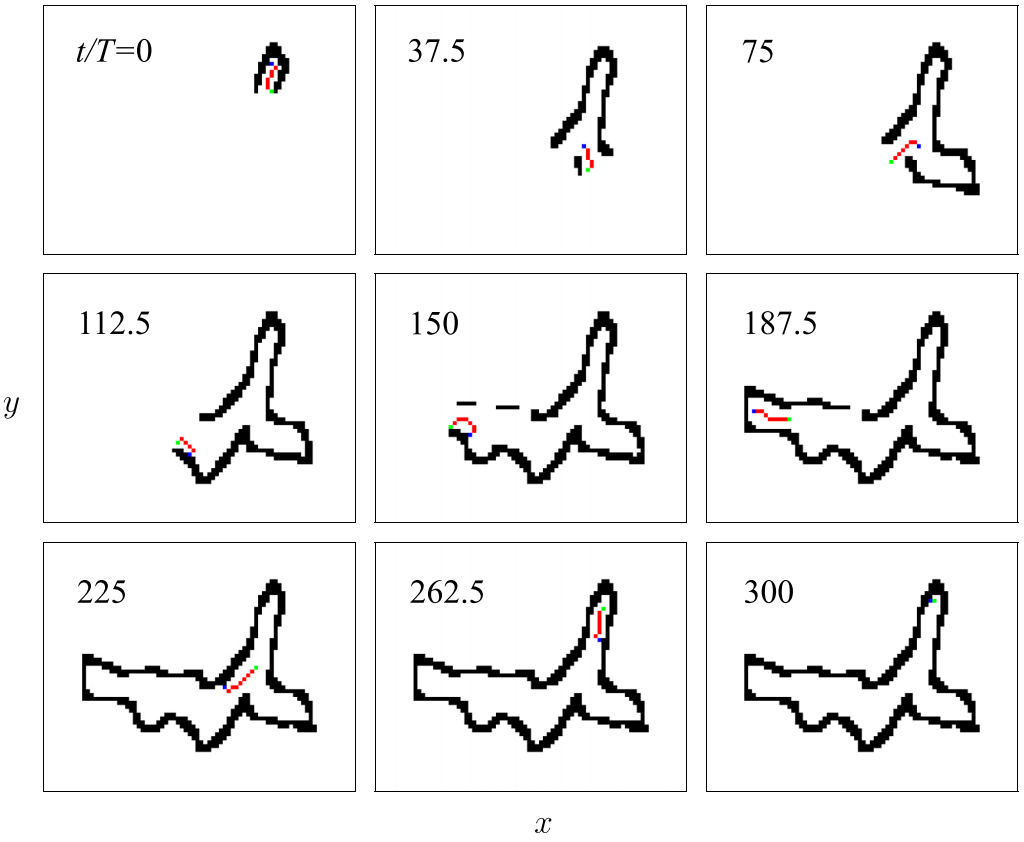


FIG. 18. Maze map constructed in real time with position of the swimmer (blue dot), planned path (red dot), pilot and target (both in green dot) at different t/T (marked at the left upper of each panel). The swimmer's task is to explore the closed boundary of the whole cave. Upon finishing the exploration, it navigates to the start point.

sensory modality for next-generation autonomous systems operating in confined, fluid-dominated environments.

Despite its contributions, this study has limitations that suggest directions for future work. Notably, our model relies solely on hydrodynamic forces and torques to sense environmental boundaries, whereas natural jellyfish such as *Tripedalia cystophora* integrates visual cues for obstacle avoidance. This discrepancy inspires the development of a multimodal agent by incorporating visual information into the state vector. A systematic comparison with vision-based reinforcement learning baselines [54] would also help quantify the relative advantages of different sensing modalities in underwater navigation tasks. Finally, as a future direction for real-world exploration, deploying multiple robots concurrently could significantly accelerate data collection.

ACKNOWLEDGMENTS

Numerical simulations were carried out on the TH-2A supercomputer in Guangzhou, China. This work has been supported by the National Natural Science Foundation of China (Grants No. 12525201, No. 12432010 and No. 12588201), and the Xplore Prize.

DATA AVAILABILITY

The data that support the findings of this article are openly available [55].

APPENDIX A: DATA AUGMENTATION

The training dataset is constructed from raw data collected during CFD simulations with randomly chosen actions. During data collection, each time step stores an extended state that contains the coordinates of the swimmer's three reference points, mass center, and target position at both the previous and current time steps,

$$\begin{aligned} \boldsymbol{s}_{t^*}^{\mathrm{raw}} = \big(&x_1^{t^*-1}, y_1^{t^*-1}, x_2^{t^*-1}, y_2^{t^*-1}, x_3^{t^*-1}, y_3^{t^*-1}, x_{sw}^{t^*-1}, y_{sw}^{t^*-1}, x_{ta}^{t^*-1}, y_{ta}^{t^*-1}, F_x^{t^*-1}, F_y^{t^*-1}, M^{t^*-1}, n^{t^*-1}, \\ &x_1^{t^*}, y_1^{t^*}, x_2^{t^*}, y_2^{t^*}, x_3^{t^*}, y_3^{t^*}, x_{sw}^{t^*}, y_{sw}^{t^*}, x_{ta}^{t^*}, y_{ta}^{t^*}, F_x^{t^*}, F_y^{t^*}, M^{t^*}, n^{t^*}\big), \end{aligned} \tag{A1}$$

where the superscripts t^*-1 and t^* denote the previous and current time steps, (x_{sw}, y_{sw}) is the swimmer's mass center, and (x_{ta}, y_{ta}) is the target position. Storing the target coordinates explicitly in the raw state is the key that enables data augmentation. The target can be shifted retroactively to generate new samples without additional CFD simulations.

Using this raw state, the variables in the agent's state vector Eq. (10) are computed as follows. Let $\boldsymbol{\xi}^{t^*} = (x_{ta}^{t^*}, y_{ta}^{t^*}) - (x_{sw}^{t^*}, y_{sw}^{t^*})$ be the vector from the swimmer to the target, and $\boldsymbol{\eta}^{t^*} = (x_2^{t^*}, y_2^{t^*}) - (x_{sw}^{t^*}, y_{sw}^{t^*})$ be the direction of the swimmer's symmetric axis. The relative velocity is

$$\left(u_1^{t^*}, u_2^{t^*}\right) = \boldsymbol{\xi}^{t^*} - \boldsymbol{\xi}^{t^*-1}. \tag{A2}$$

The distance to the target is $d^{t^*} = |\boldsymbol{\xi}^{t^*}|$. The alignment angle between the symmetric axis and the line to the target is

$$\theta^{t^*} = \arccos\left(\frac{\boldsymbol{\xi}^{t^*} \cdot \boldsymbol{\eta}^{t^*}}{|\boldsymbol{\xi}^{t^*}||\boldsymbol{\eta}^{t^*}|}\right) \cdot \mathrm{sgn}((\boldsymbol{\xi}^{t^*} \times \boldsymbol{\eta}^{t^*}) \cdot \boldsymbol{e}_z), \tag{A3}$$

where $\boldsymbol{e}_z$ is the unit vector normal to the plane. The angular velocity of the symmetric axis is the frame-to-frame change

$$\Omega^{t^*} = \theta^{t^*} - \theta^{t^*-1}. \tag{A4}$$

Note that Ω is a purely kinematic quantity computed from the stored coordinates and it does not rely on the torque M, which is recorded independently in the raw state. The hydrodynamic forces (F_x, F_y) and torque M are taken directly from the raw state.

The training dataset contains two parts. The first part contains samples computed directly from the raw dataset without modifying the target position. The second, synthetic part is generated by virtually shifting the target position in each raw sample. Concretely, for a given raw sample, a new target location $(\tilde{x}_{ta}, \tilde{y}_{ta})$ is chosen. Recomputing $\boldsymbol{\xi}, d, \theta, \Omega, (u_1, u_2)$, and the reward r with the shifted target yields a new state-action-reward tuple without running any additional CFD simulation. The synthetic samples fill regions of the state space that are underrepresented in the raw dataset and enlarge the dataset.

APPENDIX B: DETAILS ON REINFORCEMENT LEARNING AND TRAINING OF SAC

Reinforcement learning is a type of machine learning where an agent learns to make decisions by interacting with an environment to maximize cumulative rewards. The core principle is based on trial-and-error learning, where the agent takes actions in an environment, observes the outcomes, and adjusts its behavior accordingly. The tracking task is formulated as a sequential decision problem solved using reinforcement learning. The decision making is modeled as a Markov decision process where the agent makes decision solely based on the current state [1].

The trajectory of the swimmer can be written as

$$\Gamma_t = (s_0, a_0, r_0, ..., s_{t-1}, a_{t-1}, r_{t-1}, s_t, a_t, r_t), \tag{B1}$$

where s_t, a_t, and r_t denote the state, the action taken by the agent, and the reward received at a given time $t^* = t\Delta t$, respectively. In the present study, $\Delta t = T/4$, corresponding to the time interval for force application. The agent takes action a_0 at $t = 0$, and then it receives reward r_0 and its state changes from s_0 to s_1. The same process continues until time step t. The action a_t taken by the agent is determined solely by the current state s_t and the agent's policy $\pi(\Theta, \cdot)$, which is implemented as a neural network with parameters Θ. The reward function, $r_t = r_t(s_t, a_t)$, depends on the current state and action.

SAC is an offline algorithm where its training does not involve constantly interacting with the environment. Instead, it learns from history data. Offline method is preferred in this scenario because of the time cost for simulation of the flow field. It is very time-consuming to constantly interact with the environment. The training of SAC is detailed in Algorithm 1.

ALGORITHM 1. Training of SAC method.

Initialize : load dataset obtained from simulation

Initialize : policy network π_p with random parameter Θ_p

Initialize : 4 deep Q network $Q_{\phi_1}, Q_{\phi_2}, Q_{t_1}, Q_{t_2}$ with random parameter $\Theta_{\phi_1}, \Theta_{\phi_2}, \Theta_{t_1} = \Theta_{\phi_1}, \Theta_{t_2} = \Theta_{\phi_2}$

Initialize : hyper parameters learning rate ε and ε', discount factor γ, relaxation factor τ, temperature α

for $i = 1$ ***to*** N **do**

- sample $(\boldsymbol{s}_t, a_t, r_t, \boldsymbol{s}_{t+1}, D)$ tuple batch from dataset;
- calculate every tuple in the batch
- $\mathbb{E}_a\left(Q_{\phi_1}(s_t, a) - \alpha\log\left(\pi_p(a|s_t)\right)\right) = \sum_{i=1}^{N_a} \pi_p(i|s_t)\left(Q_{\phi_1}(s_t, i) - \alpha\log(\pi_p(i|s_t))\right)$;
- $\Theta_p \leftarrow \Theta_p + \varepsilon\nabla_p\mathbb{E}_a$;
- $Q_{\text{predict}_1} = Q_{\phi_1}(s_t, a_t), Q_{\text{predict}_2} = Q_{\phi_2}(s_t, a_t)$;
- $Q' = \min\left(\sum_{i=1}^{N_a} Q_{t_1}(s_{t+1}, i)\pi_p(i|s_{t+1}), \sum_{i=1}^{N_a} Q_{t_2}(s_{t+1}, i)\pi_p(i|s_{t+1})\right)$;
- $Q_{\text{target}} = r_t + \gamma(1 - D)\left(Q' - \alpha\sum_{i=1}^{N_a} \pi_p(i|s_{t+1})\log(\pi_p(i|s_{t+1}))\right)$;
- $L_1 = \frac{1}{2}\left(Q_{\text{predict}_1} - Q_{\text{target}}\right)^2, L_2 = \frac{1}{2}\left(Q_{\text{predict}_2} - Q_{\text{target}}\right)^2$;
- $\Theta_{\phi_1} \leftarrow \Theta_{\phi_1} - \varepsilon'\nabla_{\phi_1}L_1$;
- $\Theta_{\phi_2} \leftarrow \Theta_{\phi_2} - \varepsilon'\nabla_{\phi_2}L_2$;
- $\Theta_{t_1} \leftarrow (1 - \tau)\Theta_{t_1} + \tau\Theta_{\phi_1}$;
- $\Theta_{t_2} \leftarrow (1 - \tau)\Theta_{t_2} + \tau\Theta_{\phi_2}$;

APPENDIX C: A* ALGORITHM FOR NAVIGATING

The A* (A-star) algorithm is a widely used pathfinding and graph traversal algorithm [63]. It efficiently finds the shortest path between nodes in a graph by combining the actual cost from the start node with an estimated heuristic cost to the goal. This heuristic approach allows A* to explore fewer paths compared to other algorithms like Dijkstra's, making it both optimal and complete when using an admissible heuristic. A* is commonly applied in fields such as robotics, video games, and navigation systems for route planning. For the navigation in present study, obstacles are represented by Lagrangian points of their boundaries. The domain with obstacles is discretized into a grid maze where the A* algorithm is applicable.

The algorithm is detailed in Algorithm 2. The cost of a position P in the algorithm is a heuristic function

$$\text{cost}(P) = n(P) + \|\mathcal{P} - \mathcal{E}\|_1 + d(P), \tag{C1}$$

where the first term $n(P)$ is the path length from $\mathcal{S}$ to P, $\|\mathcal{P} - \mathcal{E}\|_1$ is the sum of coordinate difference between P and $\mathcal{E}$ and $d(P)$ the distance from the closest obstacle if there is obstacle point in range. This heuristic function encourages the path to be short and keep distance to obstacle at the same time.

ALGORITHM 2. A* algorithm.

Initialize : empty queue $\mathcal{Q}$, start position $\mathcal{S}$, end position $\mathcal{E}$

push $\mathcal{S}$ to $\mathcal{Q}$;
while $\mathcal{Q}$ *is not empty* **do**
    $x \leftarrow$ pop element with smallest cost from $\mathcal{Q}$;
    if x *is* $\mathcal{E}$ **then**
        backtrack from $\mathcal{E}$;
        return the trajectory from $\mathcal{S}$ to $\mathcal{E}$;
    push non-repetitive neighbor positions of x to $\mathcal{Q}$;

Figures 15 and 18 showcase the process of environment perception and path planning of the unknown environment in Sec. IV. The path consists of the grid points connecting the grid point that is closest to the swimmer to the target grid point. After the path is established, the pilot is set to the grid point on the path where there is no obstacles between it and the swimmer and its distance to the swimmer is closest to $\mathcal{R}$. One exception is that if the swimmer is close enough to the target where its distance to the target is smaller than $\mathcal{R}$, and there is no obstacle between swimmer and the target, the pilot is set to the target point. In the scenario depicted in Fig. 15, the target is known and fixed on the maze map, with the mission being to reach this target. In contrast, for the scenario in Fig. 18, no target is predefined; instead, the objective is to fully explore the boundary of the domain. Accordingly, the target is computed in real time based on the environment sensed by the swimmer. The selected target is the farthest reachable point detected within the explored region. The A* algorithm is then employed to plan a path from the swimmer's current position to this target, while a pilot is also calculated as described previously.

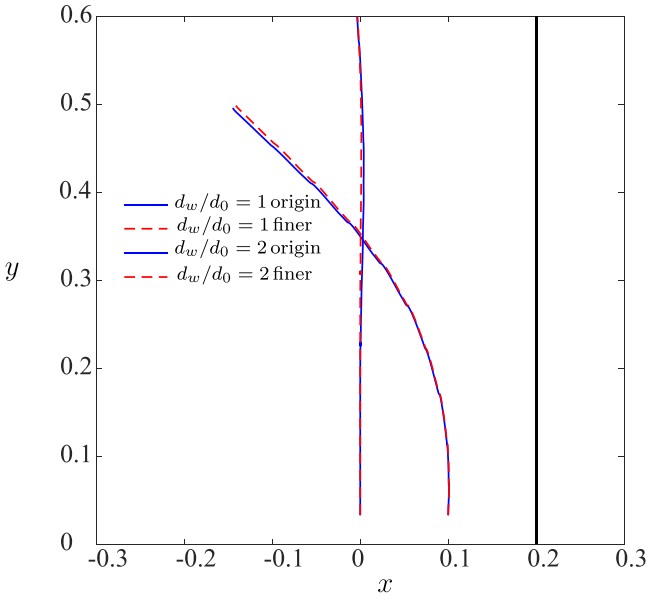


FIG. 19. Trajectories of the swimmer moving forward with a side wall (black line) at origin (blue line) and finer (red dotted line) gird resolution over $t/T = 10$. Their maximum difference is approximately 5%.

APPENDIX D: CONVERGENCE TEST

To verify grid convergence, we repeated the simulation with doubled resolution. Figure 19 compares the swimmer trajectories at the two resolutions for two initial wall distances, after aligning the initial mass-center positions. The maximum displacement between corresponding trajectories remains within approximately 5% of the swimmer diameter d_0 over the full simulation duration. This level of agreement is sufficient for the present analysis, where a practical balance between numerical accuracy and the computational cost of CFD data generation is required.

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An introduction* (MIT Press, Cambridge, MA, 2018), pp. 47, 55, 63.

[2] J. Viquerat, P. Meliga, A. Larcher, and E. Hachem, A review on deep reinforcement learning for fluid mechanics: An update, Phys. Fluids **34**, 111301 (2022).

[3] K. Ren, C. Gao, N. Xiong, and W. Zhang, Adaptive control of transonic buffet and buffeting flow with deep reinforcement learning, Phys. Fluids **36**, 016143 (2024).

[4] M. Ye, H. Ma, Y. Ren, C. Zhang, O. J. Haidn, and X. Hu, Drlinsph: An open-source platform using deep reinforcement learning and sphinxsys for fluid-structure-interaction problems, Eng. Appl. Comput. Fluid Mech. **19**, 2460677 (2025).

[5] G. Novati, S. Verma, D. Alexeev, D. Rossinelli, W. M. van Rees, and P. Koumoutsakos, Synchronisation through learning for two self-propelled swimmers, Bioinspir. Biomim. **12**, 036001 (2017).

[6] T. P. Dussauge, W. J. Sung, O. J. P. Fischer, and D. N. Mavris, A reinforcement learning approach to airfoil shape optimization, Sci. Rep. **13**, 9753 (2023).

[7] C. Lu, J. Liu, and A. Guo, Shape optimization of the floating bridge pontoons based on the Fourier series expansion and deep reinforcement learning methods, Ocean Eng. **325**, 120792 (2025).

[8] P. Scavella, G. Paolillo, and C. Greco, Deep reinforcement learning-based airfoil design and optimization: An aerodynamic analysis, Aerosp. Sci. Technol. **167**, 110638 (2025).
[9] Z. Liu, M. Zhang, D. Sun, L. Li, and G. Chen, A deep reinforcement learning optimization framework for supercritical airfoil aerodynamic shape design, Struct. Multidiscip. Optim. **67**, 34 (2024).
[10] T. Noda, K. Okabayashi, S. Kimura, S. Takeuchi, and T. Kajishima, Optimization of configuration of corrugated airfoil using deep reinforcement learning and transfer learning, AIP Adv. **13**, 035328 (2023).
[11] D. Fan, L. Yang, Z. Wang, M. S. Triantafyllou, and G. E. Karniadakis, Reinforcement learning for bluff body active flow control in experiments and simulations, Proc. Natl. Acad. Sci. USA **117**, 26091 (2020).
[12] B. Han, W. Huang, and C. Xu, Deep reinforcement learning for active control of flow over a circular cylinder with rotational oscillations, Int. J. Heat Fluid Flow **96**, 109008 (2022).
[13] M. Chatzimanolakis, P. Weber, and P. Koumoutsakos, Drag reduction in flows past 2D and 3D circular cylinders through deep reinforcement learning, Phys. Rev. Fluids **9**, 043902 (2024).
[14] T. Lee, J. Kim, and C. Lee, Turbulence control for drag reduction through deep reinforcement learning, Phys. Rev. Fluids **8**, 024604 (2023).
[15] Q. Wang, L. Yan, G. Hu, W. Chen, J. Rabault, and B. R. Noack, Dynamic feature-based deep reinforcement learning for flow control of circular cylinder with sparse surface pressure sensing, J. Fluid Mech. **988**, A4 (2024).
[16] F. Zhao, Y. Zhou, F. Ren, H. Tang, and Z. Wang, Mitigating the lift of a circular cylinder in wake flow using deep reinforcement learning guided self-rotation, Ocean Eng. **306**, 118138 (2024).
[17] G. M. Cavallazzi, L. Guastoni, R. Vinuesa, and A. Pinelli, Deep reinforcement learning for the management of the wall regeneration cycle in wall-bounded turbulent flows, Flow, Turbul. Combust. **115**, 1291 (2025).
[18] P. Becker-Ehmck, M. Karl, J. Peters, and P. van der Smagt, Learning to fly via deep model-based reinforcement learning, arXiv:2003.08876.
[19] C. Rodwell and P. Tallapragada, Physics-informed reinforcement learning for motion control of a fish-like swimming robot, Sci. Rep. **13**, 10754 (2023).
[20] F. Xie, C. Zheng, T. Ji, X. Zhang, R. Bi, H. Zhou, and Y. Zheng, Deep reinforcement learning: A new beacon for intelligent active flow control, Aerosp. Res. Commun. **1**, 11130 (2023).
[21] Y. Lai, S. Heydari, O. S. Pak, and Y. Man, Navigation of a three-link microswimmer via deep reinforcement learning, Phys. Rev. Fluids **10**, 064103 (2025).
[22] J. Lin, S. Wang, H.-D. Yao, and Y. Su, Intelligent control of the Magnus anti-rolling device: A co-simulation approach, Ocean Eng. **312**, 119304 (2024).
[23] G. Zhu, W.-Z. Fang, and L. Zhu, Optimizing low-Reynolds-number predation via optimal control and reinforcement learning, J. Fluid Mech. **944**, A3 (2022).
[24] Y. Zhi, Y. Chen, Z. Zhang, H. Zhao, and X. Deng, Artificial intelligence control of flow separation from a curved ramp, Phys. Fluids **36**, 086110 (2024).
[25] L. Biferale, F. Bonaccorso, M. Buzzicotti, P. Clark Di Leoni, and K. Gustavsson, Zermelo's problem: Optimal point-to-point navigation in 2D turbulent flows using reinforcement learning, Chaos **29**, 103138 (2019).
[26] L. Guastoni, J. Rabault, P. Schlatter, H. Azizpour, and R. Vinuesa, Deep reinforcement learning for turbulent drag reduction in channel flows, Eur. Phys. J. E **46**, 27 (2023).
[27] Y. Jiao, H. Hang, J. Merel, and E. Kanso, Sensing flow gradients is necessary for learning autonomous underwater navigation, Nat. Commun. **16**, 3044 (2025).
[28] J. Qu, Y. Xu, Z. Li, Z. Yu, B. Mao, Y. Wang, Z. Wang, Q. Fan, X. Qian, M. Zhang, M. Xu, B. Liang, H. Liu, X. Wang, X. Wang, and T. Li, Recent advances on underwater soft robots, Adv. Intell. Syst. **6**, 2300299 (2024).
[29] S. A. Hasib, M. M. Gulzar, S. R. Oishy, M. Maaruf, S. Habib, and A. Shakoor, An investigation of innovative strategies in underwater soft robotics, Eng. Sci. Technol. Int. J. **70**, 102123 (2025).
[30] W. Chen, S. Yang, C. Zhu, Y. Cheng, Y. Shi, C. Yu, and K. Liu, Scalable jet swimmer driven by pulsatile artificial muscles and soft chamber buckling, Adv. Mater. **37**, 2503777 (2025).

[31] G. Li, P. Shen, T.-W. Wong, M. Liu, Z. Sun, X. Liu, Y. Chen, X. Wang, H. Zhang, B. Hu, D. Chen, Z. Zhang, C. Zhang, R. Wang, W. Zhang, S. Nie, X. Zhang, J.-W. Wong, H. Zhou, W. Li, *et al.*, Plasticized electrohydraulic robot autopilots in the deep sea, Sci. Robot. **10**, eadt8054 (2025).

[32] Y. Xu, J. Zhuo, M. Fan, X. Li, X. Cao, D. Ruan, H. Cao, F. Zhou, T. Wong, and T. Li, A bioinspired shape memory alloy based soft robotic system for deep-sea exploration, Adv. Intell. Syst. **6**, 2300699 (2024).

[33] G. Xue, F. Bai, L. Guo, P. Ren, and Y. Liu, Research on the effects of complex terrain on the hydrodynamic performance of a deep-sea fishlike exploring and sampling robot moving near the sea bottom, Front. Mar. Sci. **10**, 1091523 (2023).

[34] P. Li, Y. Guo, H. Qin, and Z. Jiang, Hydrodynamic interaction and motion response of the autonomous underwater vehicle in turbulent flow near the underwater dune bedform, Phys. Fluids **37**, 065159 (2025).

[35] S. Terrington, M. Thompson, and K. Hourigan, Vorticity dynamics at partial-slip boundaries, J. Fluid Mech. **980**, A58 (2024).

[36] W. Zhang, X. I. A. Yang, J. Li, and M. Wan, Large-eddy simulation and modeling on the evolution of large-scale secondary vortices in turbulent boundary layer, Boundary Layer Meteorol. **191**, 14 (2025).

[37] Z. Li, Y. Zhu, Y. Wang, Y. Zhang, and L. Wang, Path following control of under-actuated autonomous surface vehicle based on random motion trajectory dataset and offline reinforcement learning, J. Ocean Eng. Sci. **10**, 724 (2024).

[38] A. Zhang, W. Wang, W. Bi, and Z. Huang, A path planning method based on deep reinforcement learning for AUV in complex marine environment, Ocean Eng. **313**, 119354 (2024).

[39] Z. Tang, X. Cao, Z. Zhou, Z. Zhang, C. Xu, and J. Dou, Path planning of autonomous underwater vehicle in unknown environment based on improved deep reinforcement learning, Ocean Eng. **301**, 117547 (2024).

[40] Y. Wang, H. Xu, H. Feng, J. He, H. Yang, F. Li, and Z. Yang, Deep reinforcement learning based collision avoidance system for autonomous ships, Ocean Eng. **292**, 116527 (2024).

[41] S. Yang, K. Wang, W. Wang, H. Wu, Y. Suo, G. Chen, and J. Xian, Dual-attention proximal policy optimization for efficient autonomous navigation in narrow channels using deep reinforcement learning, Ocean Eng. **326**, 120707 (2025).

[42] A. Hoover and L. Miller, A numerical study of the benefits of driving jellyfish bells at their natural frequency, J. Theor. Biol. **374**, 13 (2015).

[43] Z. Liu, S. Yang, and Y. Yang, Improving airfoil turbulence resilience through leading-edge undulations, AIAA J. **63**, 5344 (2025).

[44] B. J. Gemmell, J. H. Costello, S. P. Colin, C. J. Stewart, J. O. Dabiri, D. Tafti, and S. Priya, Passive energy recapture in jellyfish contributes to propulsive advantage over other metazoans, Proc. Natl. Acad. Sci. USA **110**, 17904 (2013).

[45] J. H. Costello, S. P. Colin, J. O. Dabiri, B. J. Gemmell, K. N. Lucas, and K. R. Sutherland, The hydrodynamics of jellyfish swimming, Annu. Rev. Mar. Sci. **13**, 375 (2021).

[46] N. Xu and J. O. Dabiri, Low-power microelectronics embedded in live jellyfish enhance propulsion, Sci. Adv. **6**, eaaz3194 (2020).

[47] K. B. Joshi, Modeling of bio-inspired jellyfish vehicle for energy efficient propulsion, Ph.D. thesis, Virginia Polytechnic Institute and State University, 2012.

[48] J. Frame, N. Lopez, O. Curet, and E. D. Engeberg, Thrust force characterization of free-swimming soft robotic jellyfish, Bioinspir. Biomim. **13**, 064001 (2018).

[49] P. Matharu, P. Gong, K. P. R. Guntaka, Y. Almubarak, Y. Jin, and Y. Tadesse, Jelly-Z: Swimming performance and analysis of twisted and coiled polymer (TCP) actuated jellyfish soft robot, Sci. Rep. **13**, 11086 (2023).

[50] A. Wölfl, H. Snaith, S. Amirebrahimi, C. W. Devey, B. Dorschel, V. Ferrini, V. A. I. Huvenne, M. Jakobsson, J. Jencks, G. Johnston, G. Lamarche, L. Mayer, D. Millar, T. H. Pedersen, K. Picard, A. Reitz, T. Schmitt, M. Visbeck, P. Weatherall, and R. Wigley, Seafloor mapping–The challenge of a truly global ocean bathymetry, Front. Mar. Sci. **6**, 283 (2019).

[51] M. L. Reaka-Kudla, Known and unknown biodiversity, risk of extinction and conservation strategy in the sea, in *Waters in Peril*, edited by L. Bendell-Young and P. Gallaugher (Springer, US, Boston, 2001), pp. 19–33.

[52] T. DeVries, C. Le Quéré, O. Andrews, S. Berthet, J. Hauck, T. Ilyina, P. Landschützer, A. Lenton, I. D. Lima, M. Nowicki, J. Schwinger, and R. Séférian, Decadal trends in the ocean carbon sink, Proc. Natl. Acad. Sci. USA **116**, 11646 (2019).

[53] Y. Chen and Y. Yang, Deep reinforcement learning for tracking a moving target in jellyfish-like swimming, J. Fluid Mech. **1017**, A18 (2025).

[54] F. Tonti, J. Rabault, and R. Vinuesa, Navigation in a simplified urban flow through deep reinforcement learning, J. Comput. Phys. **538**, 114194 (2025).

[55] Y. Chen and Y. Yang, Code and dataset for current study, Github, 2025, https://github.com/yygroup/prfmazerl/tree/main.

[56] C. S. Peskin, The immersed boundary method, Acta Numer. **11**, 479 (2002).

[57] W. Tong, Y. Yang, and S. Wang, Estimating thrust from shedding vortex surfaces in the wake of a flapping plate, J. Fluid Mech. **920**, A10 (2021).

[58] B. E. Griffith, R. D. Hornung, D. M. McQueen, and C. S. Peskin, An adaptive, formally second order accurate version of the immersed boundary method, J. Comput. Phys. **223**, 10 (2007).

[59] N. A. Battista, W. C. Strickland, and L. A. Miller, *IB2d:* A Python and MATLAB implementation of the immersed boundary method, Bioinspir. Biomim. **12**, 036003 (2017).

[60] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, and S. Levine, Soft actor-critic algorithms and applications, arXiv:1812.05905.

[61] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, Proximal policy optimization algorithms, arXiv:1707.06347.

[62] See Supplemental Material at http://link.aps.org/supplemental/10.1103/g3qy-d7q7 for comparison between two scenarios in high Re, tracking process in multiple obstacles, and exploration task in unknown environment.

[63] P. E. Hart, N. J. Nilsson, and B. Raphael, A formal basis for the heuristic determination of minimum cost paths, IEEE T. Sys. Sci. Cybern. **4**, 100 (1968).

[64] I. Povară, H. Mitrofan, B. P. Onac, C. Marin, E. NiŢu, D. IoniŢă, A. Tudorache, and M. Vişan, Cernei mountains: Caves conveying geothermal fluids at Băile Herculane, in *Cave and Karst Systems of Romania*, edited by G. M. L. Ponta and B. P. Onac (Springer International Publishing, Cham, 2019), pp. 213–226.

[65] J. Bielecki, S. K. Dam Nielsen, G. Nachman, and A. Garm, Associative learning in the box jellyfish *Tripedalia cystophora*, Curr. Biol. **33**, 4150 (2023).